

LA CONJETURA DE ROSETTA

— MODERNIZACIÓN AGÉNTICA DIRIGIDA POR EL DOMINIO —



La conjetura de Rosetta

Modernización agéntica dirigida por el dominio

Pierre Milet Llobet

2026

Índice

Sobre este libro	3
Por qué Rosetta	3
Prólogo. La modernización que no moderniza	6
 I Por qué el dominio y no el código	 8
1 El cementerio tiene nombres	9
2 La trampa de la relocalización	11
3 El legacy como oráculo	13
4 Fidelidad al invariante, no a la estructura	15
 II El agente en todas las plantas	 17
5 El ascensor del arquitecto, ahora con agentes	18
6 Dónde acaba el agente y empieza el arquitecto	20
7 La disciplina de procedencia	22
8 El almacén	26
El sustrato determinista	26
La máquina de estados	27
Las guardas son los gates del libro	27
Señales y observabilidad	27
Quién gobierna la máquina	28
 III Explorar antes de productizar	 29
9 El 3X aplicado a la modernización	31
10 Recuperar el dominio	33
11 Recuperar y luego triar	36
12 El mapa de contextos objetivo	38
13 El diseño táctico del núcleo	40

IV Probar que no lo rompiste	42
14 La cadena transitiva	43
15 El comportamiento como especificación ejecutable	46
16 Paridad caja negra y la ley de orden	48
V Convivir con dos sistemas	50
17 El estrangulador, honesto	51
18 Cutover sin fe	53
19 Jubilar el oráculo sin perder la referencia	55
VI De Explore a Expand	57
20 Esto es una hipótesis	58
A Antipatronos	61
B Glosario	63

Modernizar no es traducir código. Es recuperar el dominio.

THE ROSETTA CONJECTURE:

DOMAIN-DRIVEN AGENTIC MODERNIZATION

THE PENTHOUSE

Where domain is understood, value is designed, and decisions are made with confidence.

THE PRINCIPLES OF DOMAIN RECOVERY

1. MODERNIZATION IS NOT TRANSLATION

Translating code line-by-line preserves technical debt; true modernization requires recovering the hidden domain.

2. THE LEGACY SYSTEM AS "ORACLE"

Treat the running legacy system as the ultimate source of truth for business behavior.

3. FIDELITY TO INVARIANTS, NOT STRUCTURE

Preserve what the system means (business rules) while freely redesigning how it works.

BUSINESS RULES

INVARIANTS

NOT HOW BUT WHAT

THE ARCHITECT'S ELEVATOR

It's not about moving code. It's about elevating architecture.

THE STATE MACHINE OF GUARDS

A deterministic framework uses "guards" to ensure every step meets safety and equivalence standards.

PROVENANCE CATEGORIES

(Evidence Personality)

	OBSERVED	Observed directly in the Oracle's behavior Highest Trust
	INFERRED	Deduced from system structure or names Medium Trust
	ATTESTED	Confirmed by a human business expert High (but subjective) Trust

AGENTS FIND, ARCHITECTS CHOOSE

Agents extract evidence and findings; humans provide judgment and make the final design decisions.

AGENTIC EXECUTION MODEL

Agents execute with traceable intent within safety guardrails. Humans set direction and constraints.

THE ENGINE ROOM

Here lives the truth. Old code. Real behavior.

IN SUMMARY

- ✓ We elevate architecture, not code.
- ✓ The domain guides. The data speaks.
- ✓ Evidence first. Opinion second.
- ✓ Agents execute. Architects decide.
- ✓ Modernization with confidence.

★ FEWER ASSUMPTIONS. MORE EVIDENCE. BETTER ARCHITECTURE. MORE VALUE.

Sobre este libro

Por qué Rosetta

Durante más de catorce siglos nadie supo leer los jeroglíficos. Los escribas que los grababan habían muerto, y con ellos se había ido el secreto de cómo sonaban y qué decían. La escritura seguía ahí, intacta y muda. Estaba todo delante, y no se entendía nada.

La piedra de Rosetta llevaba el mismo decreto en tres escrituras, y una, el griego, todavía se sabía leer. Eso decía qué ponía. Pero saber qué decía no era saber leer la escritura muerta.

La llave fue otra lengua, viva. El egipcio no había desaparecido del todo: había seguido evolucionando hasta el copto, que aún se usaba en la liturgia de la Iglesia copta, se escribía con letras griegas y, por tanto, se podía leer. Champollion no tenía a ningún escriba que le enseñara. Tenía la forma viva y fonética de la misma lengua, y a través de ella recuperó el sonido y el sentido de la escritura muerta. La lengua que seguía viva delató el secreto de la que ya no.

Es lo que hace este método. El sistema heredado es la escritura críptica, y los que la escribieron se han ido: nadie documentó la intención, los autores se jubilaron, el secreto se fue con ellos. Pero el sistema no ha muerto del todo. Sigue corriendo, sigue comportándose, sigue hablando cada día en producción. Ese comportamiento vivo es tu copto: la forma legible que sobrevive a sus autores y que te deja recuperar el significado que se llevaron. No interrogas a los escribas. Se fueron. Interrogas a la lengua viva. A eso lo llamo el oráculo.

No traduces el código a otro lenguaje. Descifras el dominio que el código dejó de saber explicar.

Y conjetura porque eso es lo que ofrezco. No un método probado contra cientos de migraciones, sino una hipótesis razonada sobre cómo debería hacerse cuando el dominio está en el centro. Una conjetura que pide recorrerse, no creerse.

Este no es un libro sobre herramientas. Las herramientas que menciona, un compilador que reescribe un lenguaje viejo en uno nuevo, un framework que organiza el código en cortes verticales, un grafo que proyecta un sistema sobre una estructura navegable, un agente que lee, razona y escribe, todas envejecerán. Algunas estarán obsoletas antes de que termines de leer. Por eso aparecen siempre como ejemplos de una idea, nunca como la idea misma. Lo que el libro defiende es un método, y un método se mide por los principios que lo sostienen, no por las piezas que hoy lo encarnan.

El libro tiene un centro y dos ejes. El centro es el dominio. Los dos ejes son prestados. El primero es el ascensor del arquitecto de Gregor Hohpe, que va del penthouse, donde se decide la estrategia, a la sala de máquinas, donde el código se ejecuta. El segundo es el modelo 3X de Kent Beck, Explore, Expand, Extract, que describe cómo madura una idea desde que es una corazonada hasta que es un estándar. Este libro vive declaradamente en la fase Explore. No te vende un método probado contra cientos de migraciones de producción. Te ofrece un argumento, coherente y razonado, de por qué un método debería funcionar, y te invita a recorrerlo en ciclos rápidos de aprendizaje para descubrir si merece pasar a Expand.

Esa honestidad es deliberada. Un libro que sobre-afirma se desmiente con un solo encargo fallido. Un libro que argumenta desde principios resiste, porque los principios no se refutan con anécdotas. Si dentro de cinco años las herramientas de hoy son arqueología pero el argumento sigue en pie, el libro habrá hecho su trabajo.

Una última nota sobre los ejemplos. Hay un sistema al que volver capítulo tras capítulo, y se llama CardDemo. Aparece siempre en una caja aparte, separada del argumento, para que puedas seguir el hilo del caso de corrido o saltártelo sin perder el razonamiento. La primera caja lo presenta; las siguientes lo retoman allí donde cada capítulo lo ilumina.

CardDemo • paso 1/11

CardDemo es el sistema de ejemplo de modernización de mainframe que AWS publica en abierto: treinta y un programas COBOL, con sus copybooks, su batch y su CICS. No es un juguete inventado para la ocasión, es COBOL de verdad, con acoplamientos de verdad y al menos una rareza de negocio de verdad.

Gestiona las cuentas de tarjeta de crédito de un emisor. Alrededor del cliente giran tres objetos: la cuenta, que es la línea de crédito con su límite y su saldo; la tarjeta, ligada a la cuenta; y una referencia cruzada que reconcilia las tres cosas. Cada día procesa movimientos contra dos reglas que nunca deja cruzar: ninguno supera el crédito disponible del ciclo, ninguno cae sobre una cuenta caducada. Al cerrar el ciclo calcula el interés buscando la tasa en una tabla de pricing por el grupo de la cuenta.

El registro de cuenta lo declara el copybook CVACT01Y. Estos seis campos bastan para seguir el hilo:

```
01 ACCOUNT-RECORD.  
   05 ACCT-ID                PIC 9(11).  
   05 ACCT-CURR-BAL          PIC S9(10)V99.  
   05 ACCT-CREDIT-LIMIT      PIC S9(10)V99.  
   05 ACCT-CURR-CYC-CREDIT   PIC S9(10)V99.  
   05 ACCT-CURR-CYC-DEBIT     PIC S9(10)V99.  
   05 ACCT-GROUP-ID          PIC X(10).
```

Fíjate en que conviven dos cosas: el saldo arrastrado, ACCT-CURR-BAL, y la exposición del ciclo en curso, ACCT-CURR-CYC-CREDIT menos ACCT-CURR-CYC-DEBIT. Esa convivencia esconde la rareza que reaparecerá: el control de crédito mira el ciclo, no el saldo arrastrado. Puede ser intencionado o un defecto. Eso lo decide el negocio, no el código.

Y conviene recordarlo: es un ejemplo, no el sistema de producción de un banco. Lo que funciona aquí sugiere, no demuestra, que funcionará sobre un corpus mayor y más sucio.

Entidades de CardDemo alrededor del cliente

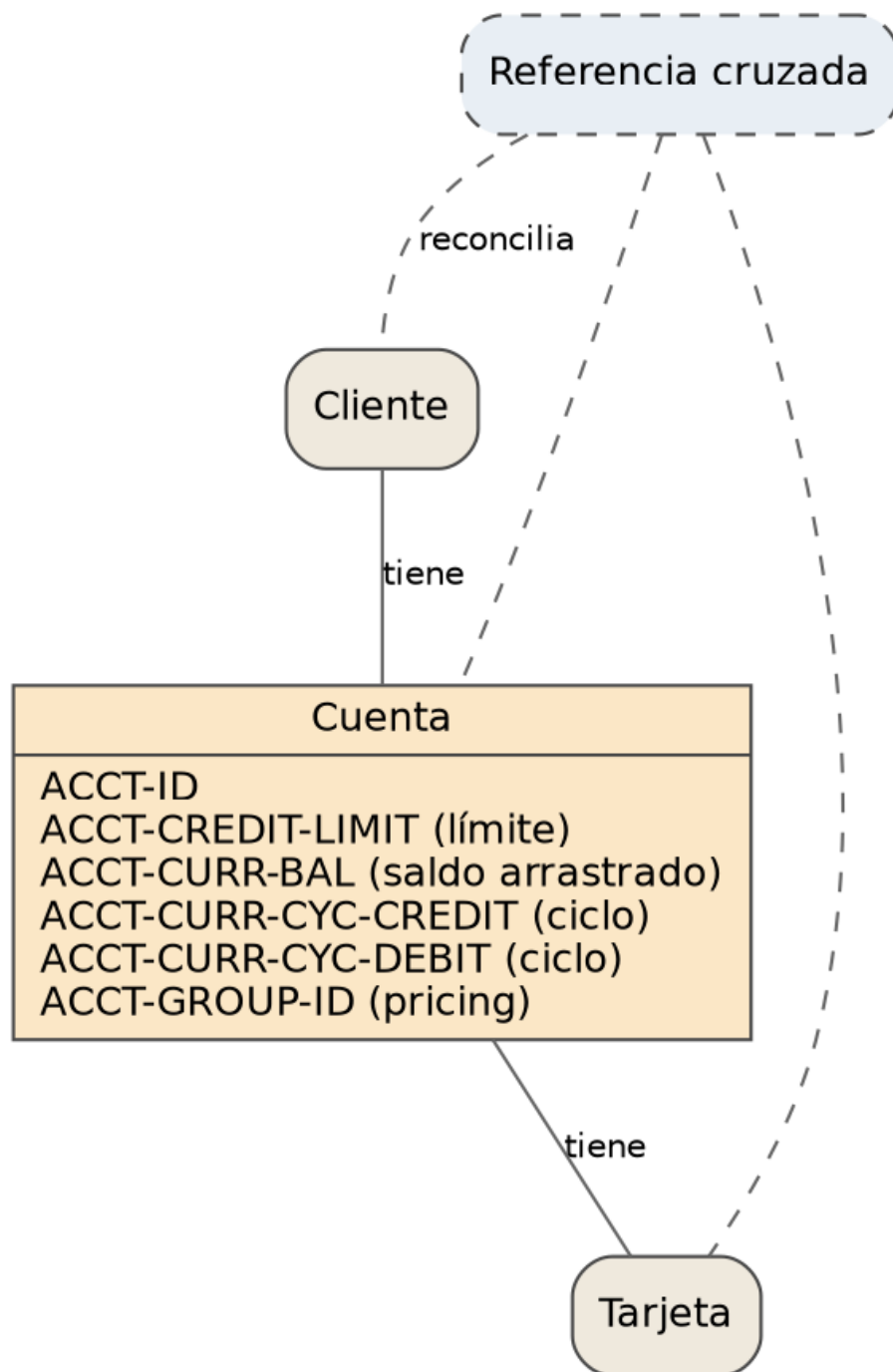


Figura. Las tres entidades de CardDemo alrededor del cliente: la cuenta con su límite, su saldo y su ciclo; la tarjeta ligada a la cuenta; y la referencia cruzada que reconcilia las tres.

Prólogo. La modernización que no moderniza

Empieza por lo que este libro no dice.

No dice «usa IA para reescribir tu COBOL». Argumenta casi lo contrario. La oferta más ruidosa de la industria ahora mismo es que los agentes traducen millones de líneas de código heredado a una velocidad sin precedentes. Esa oferta debería preocuparte, porque es el mismo pipeline sin certificar que lleva décadas fracasando, solo que más rápido. La velocidad aplicada a un método que no funciona no produce modernización. Produce el siguiente post-mortem, antes.

La primera contribución de la IA a la modernización de mainframe no es generar código. Es hacer asequible el entendimiento, en sistemas donde se había vuelto imposible. El objetivo del método de este libro no es reescribir. Es recuperar el dominio: reconstruir una comprensión verdadera de qué hacen estos sistemas, qué capacidades y reglas de negocio codifican, dónde se concentra el riesgo, dónde vive la exposición de seguridad y dónde está la oportunidad. Las decisiones estratégicas sobre sistemas que esta noche mueven pagos, pólizas y pensiones merecen tomarse con entendimiento, no con conjeturas disfrazadas de hoja de ruta. Y de casi todo lo que recuperes, decidirás racionalmente no reescribir nada.

Conviene situar por qué esto importa ahora. Décadas de rehosting, recompilación y transpilación ya migraron lo migrable: los sistemas periféricos, los manejables, aquellos cuyos autores siguen en nómina. Lo que queda sumergido es lo que esos métodos no pudieron tocar: monolitos transaccionales con décadas de parches encima, autores jubilados hace tiempo, documentación que miente. La mitad de la información crítica de las grandes organizaciones aún vive ahí, y la presión no deja de crecer: fin de soporte, licencias que suben sin retorno, regulación que exige evidencia sobre cajas negras, expertos que se van sin relevo. Lo sumergido era, hasta hace poco, lo imposible. Lo que ha cambiado es que entra en el territorio de lo posible. No de lo fácil. De lo posible.

Imagina que heredas un sistema. Lleva décadas en producción. Mueve dinero, o vidas, o cargamentos, algo que importa. Nadie que lo escribió sigue en la empresa. La documentación, si existe, miente. Y sin embargo el sistema funciona, día tras día, con una fiabilidad que ningún proyecto reciente ha igualado. Tu encargo es modernizarlo.

La primera tentación es la más razonable y la más equivocada. Consiste en tratar el problema como una traducción. Tienes código en un lenguaje viejo, quieres código en uno nuevo, así que traduces. Hoy hay herramientas que lo hacen casi solas. Un transpilador toma el lenguaje de origen y emite el de destino, línea por línea, estructura por estructura. El resultado compila. Pasa las pruebas, si las hay. Y has fracasado, aunque tardarás meses en notarlo.

Has fracasado porque has confundido el síntoma con la enfermedad. El problema de un sistema heredado no es que esté escrito en un lenguaje viejo. El problema es que su diseño, las decisiones sobre qué concepto vive dónde, qué regla protege qué dato, qué parte sabe de qué otra parte, se tomó hace décadas, se erosionó con cada parche, y nunca se volvió a mirar como un todo. Ese diseño es la deuda. Traducir el lenguaje conserva la deuda intacta. Acabas con la misma maraña, ahora en sintaxis moderna, y has gastado un presupuesto en moverla de sitio. La sintaxis es nueva. El diseño es el de siempre. Eso no es modernización. Es relocalización con buena letra.

Hay una variante más sofisticada que cae en la misma trampa. En vez de traducir, recompilas. Coges el binario conceptual del sistema viejo y lo haces correr sobre una plataforma nueva, sin tocar el código fuente. Es más barato, más rápido, menos arriesgado a corto plazo. Y sigue sin ser modernización, porque el diseño no ha cambiado. Has cambiado el suelo bajo los pies del sistema, no el sistema. Es una táctica de reubicación legítima, a veces la correcta, pero llamarla modernización es mentir sobre lo que has comprado.

Hay todavía una tercera variante, la más sofisticada y la más fácil de confundir con lo que defiende este libro. En vez de traducir o recompilar, descompone. Hay herramientas asistidas por IA que parten el monolito en dominios de negocio, agrupando programas y ficheros por sus dependencias, y refactorizan cada grupo al lenguaje nuevo. Hablan de dominios; algunas nombran DDD.¹ Y aun así se quedan cortas, porque agrupar el código existente en dominios no es lo mismo que recuperar el modelo de dominio que el código esconde. Una agrupación te dice qué programas se tocan entre sí. No te dice qué invariante protege la cuenta, ni cuál de sus reglas la pidió el negocio y cuál es un accidente de hace treinta años. Y refactorizar cada grupo conserva su diseño, igual que lo conservaba la traducción, ahora con fronteras más limpias alrededor de la misma deuda.

Este libro parte de una afirmación incómoda. **La modernización es un problema de dominio, no de código.** El código es la forma en que el dominio se expresó una vez, bajo unas restricciones que ya no existen. Modernizar de verdad es recuperar el dominio que el código esconde, entenderlo, y volver a expresarlo en una arquitectura que sirva a las restricciones de hoy. El lenguaje de destino es la parte fácil y la menos importante. El trabajo está en el dominio.

Si aceptas eso, el resto del libro se sigue. Si la modernización es un problema de dominio, necesitas las herramientas conceptuales del dominio, y la más madura que tenemos es el Diseño Dirigido por el Dominio (DDD), la disciplina de modelar el software desde el dominio del negocio y su lenguaje. Necesitas recuperar el modelo de dominio escondido en un sistema que nadie entiende del todo, lo que exige interrogar al propio sistema en vez de fiarte de su código o de su documentación. Necesitas rediseñar sin romper, lo que exige una disciplina de verificación que demuestre equivalencia en vez de asumirla. Y necesitas convivir con el sistema viejo mientras el nuevo nace, porque ningún sistema que importe se apaga de golpe.

Hay un segundo hilo en este libro, y entra ahora en escena. Mientras escribo esto, una nueva clase de herramienta ha empezado a cambiar la economía de todo lo anterior. Los agentes de IA, programas que leen, razonan sobre lo que leen, y actúan, pueden recorrer un sistema heredado a una velocidad y una escala que antes eran imposibles. Pero los agentes no cambian el método. Cambian lo que el método puede permitirse. Una recuperación de dominio que antes costaba meses de un experto escaso ahora puede costar días, lo que significa que puedes permitirte explorar, equivocarte y rehacer. Tareas que antes eran de la fase Extract, mecánicas y caras, vuelven a ser Explore, baratas y reversibles. Esa es la verdadera noticia. No que los agentes escriban código, sino que abaratan tanto el aprendizaje que cambian qué fases del trabajo puedes recorrer.

Por eso este es un libro sobre DDD agéntico aplicado a la modernización de legacy. El dominio en el centro. Los agentes en todas las plantas del ascensor, del penthouse a la sala de máquinas, aumentando al arquitecto en cada nivel. Y una pregunta que recorre todo el texto y le da subtítulo. Dónde acaba el agente y empieza el arquitecto. Esa frontera, vas a verla, no es una cuestión de comodidad ni de confianza. Es una decisión de arquitectura, quizá la más importante que tomarás.

Antes de teorizar por qué la traducción fracasa, conviene mirar a quienes ya lo intentaron. El cementerio tiene nombres.

¹El ejemplo más visible al escribir esto es AWS Transform for Mainframe, previsto como capacidad de Amazon Q Developer en re:Invent 2024 y en disponibilidad general desde mayo de 2025. Descompone el monolito en dominios de negocio a partir de un grafo de dependencias y refactoriza COBOL a Java, y lista el Diseño Dirigido por el Dominio como una de varias estrategias de descomposición. La industria se mueve rápido, así que conviene reverificar el estado de cualquier herramienta concreta antes de fiarse de esta descripción.

Parte I

Por qué el dominio y no el código

Capítulo 1

El cementerio tiene nombres

El prólogo argumentó por qué traducir fracasa. Este capítulo pone nombres, porque el cementerio es real, es reciente, y sus lápidas se leen.

En abril de 2018, TSB migró a más de cinco millones de clientes desde los sistemas que le alojaba Lloyds a una plataforma nueva, en un único fin de semana, declarada lista sin la evidencia para probarlo. La migración de datos fue un éxito. El sistema no. Cerca de 1,9 de sus 5,2 millones de clientes quedaron bloqueados fuera de sus cuentas, algunos vieron las cuentas de otros, y el regulador británico acabó imponiendo una multa de 48,65 millones de libras por fallos de gobierno y de gestión del riesgo, no de hardware. La lección cabe en una frase. **«La paridad de datos no es la paridad de comportamiento.»** Mover los datos intactos no garantiza nada sobre lo que el sistema hace con ellos.

En 2010, el sistema de nóminas de Queensland Health entró en producción con defectos conocidos y sin vuelta atrás. Unos 78.000 empleados cobraron de menos, de más o no cobraron, durante meses. El coste de remediarlo a mano se estimó en torno a 1.200 millones de dólares australianos a lo largo de años, y una comisión de investigación lo situó entre los peores fracasos de administración pública del país. No fue un fallo de cálculo aislado. Fue cortar sin haber demostrado que el sistema nuevo se comportaba como debía, y sin una salida cuando se vio que no.

En 2023, un gran banco europeo completó la migración de doce millones de clientes de su filial a su propia plataforma. Migración «completada». Después vinieron meses de cuentas bloqueadas, domiciliaciones rechazadas y clientes sin acceso a sus fondos, hasta que el supervisor calificó las disrupciones de inaceptables e instaló un vigilante especial dentro del banco para forzar la remediación. «Completado» no es lo mismo que «funciona». Otra vez, la paridad de datos no era la paridad de comportamiento.

Y luego está el veredicto. En 2023, el órgano independiente de auditoría informática del Estado neerlandés revisó una conversión fabril de COBOL a Java en el organismo que paga las prestaciones de desempleo e incapacidad del país, construida sobre transpilación comercial de última generación.¹ El programa la había detenido, y los auditores concluyeron que detenerla fue lo correcto, porque el enfoque no producía software mantenible. Tómame el veredicto en serio. Transpilar COBOL a un Java con forma de COBOL aporta casi nada: pagas una migración y recibes el mismo sistema, más difícil de mantener, en un lenguaje que queda bien en la diapositiva pero que ni el especialista en COBOL ni el especialista en Java pueden sostener. Si falta el entendimiento, ningún cambio de sintaxis lo suministra.

Estos cuatro no comparten un proveedor ni una tecnología. Comparten una raíz, y no es técnica. Es de gobierno. Alguien afirmó que el sistema estaba listo, o que era equivalente, en vez de demostrarlo, y cortó sobre esa afirmación. La realidad cobró la diferencia. Por eso el cementerio importa para este libro: el pipeline sin certificar que la industria quiere ahora acelerar con agentes es el mismo pipeline que llenó estas tumbas. Y la velocidad no salva a un método que no funciona; solo entierra más rápido.

¹El programa OpenVMS del UWV neerlandés (2020-2023) convertía a Java el código COBOL de dos sistemas de prestaciones mediante una herramienta comercial de conversión automática, con Capgemini y el subcontratista Blu Age. El Adviescollege ICT-toetsing (AcICT), el órgano independiente de auditoría TI del Estado, concluyó en su dictamen del 21 de diciembre de 2023 que el UWV había hecho bien en detener la conversión fabril, porque no conducía a software mantenible. Fuentes públicas de los cuatro casos: las resoluciones de la FCA y la PRA (TSB, 2022); el informe de la comisión de investigación Chesterman (Queensland Health, 2013); los comunicados de BaFin (Postbank/Deutsche Bank, 2023); y el dictamen citado de AcICT (UWV, 2023).

La salida del cementerio no es un transpilador más veloz. Es dejar de fiarse de afirmaciones, de documentación y de memoria, y empezar a interrogar al único testigo que no miente, el sistema vivo. Pero antes de eso, conviene entender con precisión por qué la traducción falla, porque el fallo es sutil y seductor. Ese es el siguiente capítulo.

Capítulo 2

La trampa de la relocalización

El código no es la especificación. Es la evidencia.



Conviene mirar de cerca por qué traducir falla, porque el fallo es sutil y seductor. Un transpilador hace exactamente lo que promete. Toma una estructura del lenguaje de origen y produce su equivalente en el de destino. Un bucle se convierte en un bucle. Una condición en una condición. Una rutina en un método. El mapeo es fiel, y ahí está el problema. La fidelidad estructural es precisamente lo que no quieres conservar.

Piensa en qué hay dentro de un programa heredado típico. Hay lógica de negocio, las reglas que definen qué significa el sistema, por qué un importe se rechaza, cuándo una operación es válida. Y hay mecanismo, la fontanería que hace funcionar esa lógica sobre la plataforma de su época, cómo se abre un fichero, cómo se pasa un bloque de memoria de un programa a otro, cómo se controla un error de entrada y salida. En el sistema viejo, lógica y mecanismo están entrelazados, porque la plataforma de la época obligaba a entrelazarlos. No había otra manera.

Cuando traduces línea por línea, traduces los dos juntos. El mecanismo viejo, que solo existía para satisfacer a una plataforma que estás abandonando, viaja al sistema nuevo disfrazado de lógica. Acabas con código moderno que abre ficheros como si todavía estuviera en la plataforma vieja, que pasa bloques de memoria entre módulos como si la memoria fuera escasa, que controla errores con códigos numéricos que ya no significan nada. Has importado la fontanería de una casa que has demolido. Y como ahora está en sintaxis moderna, es más difícil de detectar, porque parece código nuevo legítimo.

El diseño sufre el mismo destino. Si el sistema viejo metía tres responsabilidades distintas en un mismo módulo porque crearlos era caro, la traducción produce un módulo nuevo con las tres responsabilidades. Si dos conceptos que deberían estar separados compartían una estructura de datos por ahorrar, la siguen compartiendo. Las fronteras equivocadas se preservan con perfecta fidelidad. El acoplamiento se conserva. La cohesión ausente sigue ausente. Lo único que ha cambiado es el color de la sintaxis.

Aquí es donde mucha gente bienintencionada se defiende. Dicen, de acuerdo, traducimos primero y refactorizamos después. Primero un equivalente fiel, luego lo limpiamos. Suena prudente. Es una trampa más profunda, por dos razones.

La primera es económica y psicológica. Una vez que tienes un sistema nuevo que compila y funciona, el presupuesto para la limpieza desaparece. El proyecto se declara terminado. La maraña traducida se queda, ahora bendecida como base de código moderna, y los siguientes diez años de mantenimiento la tratarán como sagrada. El refactor que iba a venir después no viene nunca. Lo que iba a ser un paso intermedio se vuelve el destino.

La segunda razón es más técnica y más interesante. Refactorizar exige saber qué comportamiento debes preservar mientras cambias la estructura. Si has traducido sin entender el dominio, no tienes ese conocimiento. No sabes cuál de esas tres responsabilidades del módulo es lógica de negocio que debe sobrevivir y cuál es mecanismo que debe morir. No tienes una red de seguridad que te diga si tu limpieza ha roto algo. Refactorizar a ciegas sobre código que no entiendes no es modernización, es ruleta. Volveremos a esto con cuidado, porque la red de seguridad, la prueba de que no has roto nada, es uno de los pilares del método.

La recompilación, la otra vía, es honesta a su manera y por eso menos peligrosa, siempre que no mientas sobre ella. Hacer correr el sistema viejo sobre una plataforma nueva sin tocar el código es una reubicación. Tiene usos legítimos. Te saca de un hardware que se muere, te quita una dependencia de un proveedor, te compra tiempo. Lo que no hace es modernizar, porque el dominio sigue expresado exactamente igual, con las mismas fronteras equivocadas y el mismo mecanismo entretejido. El error no es recompilar. El error es creer que has terminado. Y conviene adelantar algo que el libro defenderá más adelante: la recompilación tiene dos usos plenamente legítimos, como instrumento, el oráculo del capítulo tres, y como destino deliberado para lo que decidas no rediseñar, el triaje del capítulo once. Lo ilegítimo no es recompilar. Es venderlo como modernización del núcleo.

Hay un principio que conviene fijar desde ya, porque va a reaparecer. **Cambiar la plataforma y cambiar el diseño son dos saltos distintos, y nunca deben darse a la vez.** Si recompilas para cambiar de plataforma, hazlo como un movimiento limpio y aislado, sin tocar el diseño. Si rediseñas, hazlo sobre una plataforma estable, sin moverla bajo tus pies. Quien intenta los dos saltos en el mismo movimiento se queda sin punto de apoyo para diagnosticar qué ha fallado cuando algo falle, y algo fallará. Este doble salto prohibido es uno de los errores más caros y más comunes, y tiene su propio capítulo más adelante.

Entonces, si no es traducir ni recompilar, qué es modernizar. La respuesta corta es recuperar el dominio y volver a expresarlo. La respuesta larga ocupa el resto del libro. Pero hay un giro previo, casi un cambio de actitud, sin el cual nada de lo demás funciona. Tienes que dejar de ver el sistema viejo como un lastre del que escapar y empezar a verlo como la fuente de verdad más fiable que tienes. Ese es el siguiente capítulo.

Capítulo 3

El legacy como oráculo

El sistema heredado tiene mala fama, y en parte se la ha ganado. Es rígido, opaco, caro de mantener, imposible de contratar gente que lo conozca. Pero hay una cosa que ese sistema hace mejor que cualquier documento, cualquier experto y cualquier especificación que puedas escribir sobre él. Sabe exactamente lo que hace. Lo demuestra cada día, en producción, sin equivocarse en los casos raros que tú ni siquiera sabes que existen.

Esa es la idea que lo cambia todo. **El sistema viejo no es solo un lastre. Es un oráculo.** Un oráculo, en el sentido que le damos aquí, es una fuente que responde con verdad a una pregunta sobre comportamiento. Le das una entrada, te da la salida correcta, correcta por definición, porque es lo que el sistema de verdad hace y lo que el negocio lleva décadas aceptando como correcto. No tienes que creerle a nadie sobre cómo se calcula un recargo en el caso límite, o qué pasa cuando dos operaciones llegan en el orden improbable. Se lo preguntas al sistema y te lo dice.

Esto reordena las prioridades de la recuperación. La tradición dice que para entender un sistema viejo lees su código y su documentación. El código es la mejor de las dos fuentes, pero es traicionero, porque mezcla lógica y mecanismo, porque está lleno de rutas muertas que ya no se ejecutan nunca, y porque leerlo te dice lo que el sistema parece hacer, no lo que hace. La documentación es peor, porque describe lo que alguien quiso que el sistema hiciera en un momento dado, y el sistema lleva años divergiendo de esa intención sin que nadie actualizara el documento. Las dos fuentes te dan hipótesis. El oráculo te da hechos.

El cambio práctico es enorme. En vez de discutir durante semanas qué significa una rutina ininteligible, la ejecutas con entradas elegidas y observas. En vez de confiar en que un experto recuerda bien la regla del caso especial, generas el caso especial y miras la respuesta. El sistema viejo se convierte en un laboratorio. Le haces experimentos. Las preguntas que antes producían reuniones ahora producen observaciones.

Para que el oráculo sea útil de verdad necesita dos cosas, y aquí es donde entran las herramientas, como ejemplos de un concepto. La primera es que responda rápido. Un oráculo que tarda una noche en darte una respuesta no sirve para explorar, porque la exploración necesita ciclos cortos, preguntar, observar, ajustar la pregunta, volver a preguntar, decenas de veces al día. Por eso interesa tener una copia del sistema viejo que corra en local, en milisegundos, no en el mainframe de producción con su cola de trabajos. Una manera de conseguirlo es recompilar el sistema viejo a una plataforma moderna que puedas ejecutar en tu propia máquina, en un contenedor, las veces que quieras. El compilador que hace esa recompilación es un ejemplo de herramienta. El concepto durable es el oráculo rápido y local, una réplica fiel del comportamiento viejo que responde en el tiempo de un ciclo de pensamiento.

Fíjate en algo elegante. La recompilación, que en el capítulo anterior era una falsa modernización si la confundías con el destino, aquí es exactamente la herramienta correcta para un propósito distinto. No estás recompilando para entregar un sistema. Estás recompilando para construirte un oráculo. La misma técnica, despreciada como meta, es valiosa como instrumento. La diferencia está en para qué la usas.

La segunda cosa que necesita el oráculo es fidelidad. Tiene que comportarse como el original, no parecerse. Si tu réplica diverge del sistema real en los casos raros, te estará mintiendo precisamente donde más vas a confiar en ella. Por eso la fidelidad de la réplica no se asume, se valida, comparándola con el sistema real sobre suficientes casos. Un oráculo no verificado es un rumor con buena reputación.

Hay un matiz honesto que conviene no esconder. El oráculo solo sabe lo que se le pregunta. Conoce el comportamiento sobre las entradas que le das, no sobre las que no se te ocurren. Si una ruta del sistema viejo no se ejecuta nunca en tus experimentos ni en el tráfico real que observas, su comportamiento queda sin capturar, y es comportamiento que puede importar el día menos pensado. El oráculo no elimina la incertidumbre, la acota y la hace visible. Esto tendrá consecuencias cuando hablemos de qué entradas elegir para interrogarlo, porque elegir bien las preguntas es la diferencia entre un oráculo que cubre las fronteras donde viven los errores y uno que solo confirma lo que ya pasa todos los días.

Por ahora quédate con el giro de actitud. El sistema que te pidieron sustituir es, mientras viva, tu mejor profesor. Antes de jubilarlo tienes que aprender de él todo lo que vas a necesitar para siempre, porque cuando se apague, esa fuente de verdad desaparece. Buena parte del método consiste en exprimir al oráculo con disciplina mientras todavía está encendido.

Capítulo 4

Fidelidad al invariante, no a la estructura

Si el sistema viejo es el oráculo y lo que persigues es preservar su comportamiento, surge una pregunta tramposa. Preservar qué, exactamente. Porque si la respuesta fuera preservarlo todo, no habría modernización posible, estarías obligado a conservar hasta el último detalle del sistema viejo, incluida la maraña que querías eliminar. La fidelidad mal entendida te devuelve a la trampa de la traducción.

La respuesta correcta exige una distinción que es el corazón conceptual de este capítulo y una de las claves de todo el método. Hay dos cosas muy distintas que un sistema viejo contiene, y solo una de ellas es sagrada.

La primera es el qué. Las reglas del negocio, los invariantes, las reglas que deben cumplirse siempre, la semántica de cada operación. Un importe nunca puede superar cierto límite. Una operación sobre una entidad caducada se rechaza. Cierta operación redondea de cierta manera y el resultado importa hasta el último decimal porque es dinero. Esto es el espacio del problema. Existe independientemente de cómo el sistema viejo decidió implementarlo. Sobreviviría aunque reescribieras el sistema de cero en cualquier lenguaje. Esto es lo que debes preservar con fidelidad absoluta.

La segunda es el cómo. La estructura concreta que el sistema viejo eligió para cumplir esas reglas. Qué módulo contenía qué lógica, qué estructura de datos compartían qué conceptos, en qué orden se hacían las cosas, dónde estaban dibujadas las fronteras. Que un mismo dato lo reescriban varios programas distintos es mecanismo de la solución vieja: el negocio nunca pidió varios escritores, solo pidió que el dato cuadre. Esto es espacio de la solución, y es una solución vieja, tomada bajo restricciones que ya no existen. No es sagrada. Es, de hecho, justo lo que vienes a cambiar.

CardDemo • paso 2/11

En CardDemo, el fichero de cuentas lo reescriben cuatro programas distintos, dos por lotes y dos en línea:

```
* CBTRN02C   posting de transacciones (batch)
* CBACT04C   cálculo de interés          (batch)
* COACTUPC   alta y edición de cuenta (CICS)
* COBIL00C   pago de factura             (CICS)
REWRITE FD-ACCTFILE-REC FROM ACCOUNT-RECORD
```

El negocio nunca pidió cuatro escritores; solo pidió que el saldo cuadre. Los cuatro escritores son mecanismo de la solución vieja, no semántica del dominio. Ese reparto de la escritura es el hotspot que reaparecerá cuando recuperemos el mapa.

Aquí está la frase que conviene grabar. **La fidelidad es a los invariantes y a la semántica, no a la estructura.** Preservas lo que el sistema significa. Eres libre de rediseñar cómo lo consigue. Esa libertad no es un permiso menor, es el premio entero de la modernización. Sin ella estarías condenado a copiar el diseño viejo. Con ella puedes redibujar fronteras, separar lo que estaba mezclado, dar a cada concepto su lugar, y aun así garantizar que el sistema sigue significando lo mismo.

Una imagen ayuda. Ningún mapa del mundo es fiel a todo a la vez. La proyección de Mercator es fiel a los rumbos, la línea de rumbo constante sale recta, y por eso sirve para navegar; a cambio deforma el área, y Groenlandia parece tan

grande como África siendo catorce veces menor. Una proyección equivalente es fiel a la superficie y deforma las formas. No es un defecto de los cartógrafos. Es imposible: una esfera no se aplanan sin romper algo, y eso lo demostró Gauss. Todo mapa plano elige a qué ser fiel y deja que lo demás se deforme.

Modernizar es hacer esa elección. Eliges ser fiel a la semántica, a lo que el sistema significa, y dejas que la estructura cambie de forma. El sistema viejo era un mapa fiel a una máquina vieja, como Mercator lo es a la navegación. El sistema nuevo será otro mapa del mismo dominio, fiel a otra cosa. El mundo, el negocio, es el mismo. Lo que cambia es a qué eres fiel al dibujarlo.

Esta distinción tiene una consecuencia que parece sorprendente y que más adelante será un pilar técnico. Si lo que preservas es la semántica y no la estructura, entonces dos sistemas pueden ser equivalentes en comportamiento y completamente distintos por dentro. El sistema nuevo puede tener otras fronteras, otros módulos, otro orden, y aun así dar la misma respuesta a la misma pregunta. Eso significa que la prueba de que no has roto nada no puede consistir en comparar las tripas de los dos sistemas, porque por diseño son distintas. Tiene que consistir en comparar lo que se observa desde fuera, el comportamiento, no la implementación. Esa es la semilla de la paridad caja negra, que es como llamaremos a verificar la equivalencia mirando solo entradas y salidas observables, nunca el interior. Lo desarrollaremos entero en la Parte IV.

La distinción también ilumina dónde se esconden los errores más peligrosos de una modernización. Hay piezas de un sistema viejo que parecen mecanismo pero llevan dentro una decisión de negocio. El ejemplo clásico es el control de un error técnico. Cuando el sistema viejo no encuentra un dato y reacciona de cierta manera, ¿esa reacción es pura fontanería, detectar una ausencia, o es una regla de negocio disfrazada, decidir que una ausencia equivale a cierto valor por defecto con consecuencias reales? Si lo tratas como mecanismo y lo descartas, has perdido silenciosamente una regla. Si lo tratas como regla y lo conservas, arrastras fontanería. Distinguir cuál es cuál no se puede hacer solo leyendo, porque el código entretiene las dos cosas. Se hace preguntándole al oráculo qué consecuencia de negocio tiene ese comportamiento. La pérdida silenciosa de semántica, conservar la cáscara técnica y perder la regla que escondía, es uno de los antipatrones más caros, y aparece en su apéndice.

Conviene cerrar con una imagen que resume las tres ideas de esta primera parte. Piensa en el dominio recuperado como un modelo que vas a poder vestir con muchas arquitecturas distintas. El modelo, las reglas y la semántica, es uno y es estable. Las arquitecturas posibles sobre ese modelo son muchas. La traducción fracasa porque copia una arquitectura vieja sin recuperar el modelo. El oráculo te permite recuperar el modelo con hechos en vez de rumores. Y la fidelidad bien entendida te dice qué del modelo es intocable, la semántica, y qué es libre, la estructura. Con eso en la mano, la pregunta deja de ser cómo traduzco este código y pasa a ser cuál de las muchas arquitecturas posibles sobre este dominio sirve mejor a las restricciones de hoy. Esa es una pregunta de arquitecto. Y los arquitectos, resulta, ahora trabajan acompañados de agentes.

Parte II

El agente en todas las plantas

Capítulo 5

El ascensor del arquitecto, ahora con agentes

Gregor Hohpe propuso una metáfora que vale para entender qué hace un arquitecto, y que aquí vamos a estirar un poco más allá de su propósito original. Imagina una organización como un edificio. En el penthouse se decide la estrategia, hacia dónde va el negocio, qué importa. En la sala de máquinas, muchas plantas más abajo, el código se ejecuta y las cosas pasan de verdad. Entre medias hay plantas intermedias, la arquitectura, los equipos, los procesos. La mayoría de la gente vive en una sola planta. Los del penthouse hablan de visión y no saben qué hay en los binarios. Los de la sala de máquinas conocen cada bit y no saben por qué importa. El valor del arquitecto, dice Hohpe, es que monta en el ascensor. Sube y baja. Traduce la estrategia en estructura y la estructura en estrategia. Conecta plantas que de otro modo no se hablarían.

En una modernización, el ascensor es literal. En el penthouse están las preguntas del dominio, qué conceptos tiene este negocio, qué reglas lo gobiernan, dónde están las fronteras naturales entre sus partes. En la sala de máquinas está el sistema viejo, su código, sus estructuras de datos, sus rutinas entretejidas. Y el trabajo de modernizar es precisamente un viaje de ascensor, bajar a la sala de máquinas a recuperar lo que el sistema hace, subir al penthouse a entender qué significa eso en términos de dominio, parar en la planta intermedia a decidir la arquitectura, y volver a bajar a verificar. Quien solo sabe leer código no puede modernizar, porque no ve el dominio. Quien solo sabe de dominio tampoco, porque no puede recuperar nada de un sistema que no sabe leer. La modernización es un oficio de ascensor.

Aquí entra el segundo hilo del libro, y conviene decirlo con precisión para que no envejezca. Ha aparecido una clase de herramienta, los agentes, que puede recorrer ese ascensor. Un agente, en el sentido que importa aquí, es un sistema capaz de leer una gran cantidad de material, razonar sobre él, y producir resultados, código, modelos, análisis, propuestas. No es magia y no es un colega. Es una herramienta con una propiedad nueva e importante. Opera en todas las plantas. Puede leer código en la sala de máquinas. Puede proponer fronteras de dominio en el penthouse. Puede generar el andamiaje de una arquitectura en las plantas intermedias. Antes, cada planta exigía una herramienta distinta y una persona distinta. Ahora hay una clase de herramienta que las recorre todas.

Esto es lo que justifica el adjetivo agéntico del título, y conviene entender bien por qué. No es que los agentes sean un acelerador puntual de una tarea concreta, como un compilador más rápido. Es que los agentes están presentes en cada nivel del trabajo, aumentando al arquitecto en todas las plantas del ascensor a la vez. El arquitecto que antes bajaba solo a la sala de máquinas ahora baja con una herramienta que lee diez veces más rápido y no se cansa. El que subía al penthouse a proponer un mapa de contextos delimitados (bounded contexts) ahora sube con una herramienta que ya ha leído todo el sistema y trae candidatos. El DDD sigue en el centro, intacto, porque el DDD es la disciplina de poner el dominio primero, y eso no lo inventaron los agentes ni lo van a sustituir. Lo que cambia es que ahora el dominio se recupera, se modela y se verifica con un acompañante en cada planta.

Hay que resistir dos errores opuestos sobre esto. El primero es el desdén, creer que los agentes son autocompletado con ínfulas y que el método no cambia en nada. El segundo es la fascinación, creer que los agentes hacen el trabajo y que el arquitecto sobra. Los dos errores fallan por el mismo sitio, no entienden qué cambia exactamente. Lo que cambia no son los principios, es la economía. Y cuando la economía de una actividad cambia lo suficiente, cambia qué actividades te puedes permitir, y eso sí altera el método, no en sus principios sino en su alcance.

El ejemplo más claro lo da el propio modelo 3X de Beck, que veremos en detalle más adelante. Beck distingue tres fases

en la vida de una idea. Explore, cuando buscas qué funciona y casi todo lo que pruebas se descarta. Expand, cuando algo funciona y hay que escalarlo. Extract, cuando ya es estable y exprimes su eficiencia. En una modernización tradicional, recuperar el dominio se trataba como si fuera Extract: un acto final, caro, lento, hecho por expertos escasos, que hacías una vez y dabas por bueno aunque fuera malo, porque rehacerlo era impensable. Con un agente que recupera en días lo que antes costaba meses, esa misma tarea baja de coste hasta volverse Explore. Te puedes permitir recuperar el dominio, mirarlo, no convencerte, tirarlo y volver a recuperarlo de otra manera. Lo que era un compromiso irreversible se vuelve un experimento barato. Esa es la transformación profunda. No que el agente recupere el dominio, sino que lo abarata tanto que recuperarlo deja de ser un acto único y solemne y se convierte en un ciclo de aprendizaje rápido.

De modo que el agente monta en el ascensor con el arquitecto, en todas las plantas, y abarata el viaje hasta el punto de cambiar qué viajes te puedes permitir. Pero hay una planta donde el agente no decide, solo propone. Saber cuál es esa frontera, y por qué tiene que existir, es el siguiente capítulo, y es la decisión más importante del libro.

Capítulo 6

Dónde acaba el agente y empieza el arquitecto

Si el agente recorre todas las plantas, surge la pregunta natural y peligrosa. Entonces, qué hace el arquitecto que el agente no haga. La respuesta perezosa es lo que el agente no sabe hacer todavía, y es perezosa porque envejece con cada versión nueva del agente y porque trata la frontera como una limitación técnica temporal. La respuesta correcta es distinta y no envejece. Hay una clase de decisiones que el arquitecto debe tomar no porque el agente no pueda, sino porque no debe delegarse, y entender por qué es entender el método.

Empecemos por cómo razona un agente, porque ahí está la clave. Un agente no razona bien sobre código fuente crudo. El código es largo, repetitivo, lleno de detalle irrelevante, y entreteje lógica con mecanismo. Lo que un agente hace bien es razonar sobre proyecciones del sistema, representaciones que destilan una dimensión del sistema y descartan el ruido. Un grafo que muestra qué llama a qué. Un modelo que muestra qué dato fluye hacia qué decisión. Una estructura que muestra qué reglas protegen qué entidad. Sobre esas proyecciones el agente razona con potencia, porque la proyección ya ha hecho el trabajo de separar la señal del ruido. El código es el territorio, y la proyección es el mapa. Más adelante daremos un paso atrás y veremos que el propio código es también un mapa, el que sus autores dibujaron de un territorio más hondo, el negocio; por ahora basta este nivel. Y los agentes, como los exploradores, trabajan mejor con mapas.

Esto ya dibuja una primera frontera. Construir buenas proyecciones, decidir qué dimensión del sistema merece un mapa, es trabajo de arquitecto. El agente navega el mapa, pero qué mapa dibujar para qué pregunta es una decisión de diseño. Un mapa equivocado lleva a un agente potentísimo a una conclusión equivocada con total seguridad, que es la peor clase de error.

Pero la frontera de fondo es más profunda y tiene que ver con la diferencia entre hallar y elegir. Cuando recuperas un dominio de un sistema viejo, hay cosas que descubres y cosas que decides. Descubres que existe cierta regla, porque está ahí, en el comportamiento del oráculo, y la puedes señalar. Eso es un hallazgo. Tiene una fuente. Lo puede hacer un agente, porque consiste en observar y reportar lo que se observa. En cambio, cuando decides que dos conceptos que el sistema viejo mezclaba deben ir en contextos separados, eso no lo has hallado en ninguna parte. El sistema viejo no te lo dijo. Lo has elegido, a la luz de tu juicio sobre qué sirve mejor al negocio de hoy. Eso es una decisión de diseño. No tiene una fuente en el sistema viejo, porque su fuente eres tú.

La frontera entre el agente y el arquitecto es la frontera entre hallar y elegir. El agente halla. Reporta lo que el sistema hace, con su fuente, porque hallar es observar. El arquitecto elige. Decide qué arquitectura, qué fronteras, qué reglas consolidar o separar, porque elegir es un acto de juicio que no se deriva de ninguna observación. Y la razón por la que esta frontera no debe cruzarse no es que el agente sea incapaz de proponer una arquitectura, claro que puede proponerla. La razón es que una propuesta sin fuente, presentada como si fuera un hallazgo, contamina el modelo con decisiones disfrazadas de hechos. Y un modelo donde no puedes distinguir qué recuperaste del sistema de qué inventaste tú es un modelo en el que no puedes confiar para nada importante, porque no sabes cuánto de él es verdad sobre el negocio y cuánto es opinión de una herramienta.

Hay un viejo contraste que da nombre a esta frontera. Aristóteles ponía el conocimiento en lo observable, en lo que se

ve hacer al mundo. Platón lo ponía en las ideas, en lo que las cosas deben ser, que no se lee en ningún sitio porque no está ahí fuera, se alcanza pensando. El oráculo es aristotélico: dice lo que el sistema hace, y lo dice porque se observa. El arquitecto, cuando elige, es platónico: decide lo que el sistema debe ser, y eso no lo encuentra en el código ni en el oráculo, no porque no haya mirado bien, sino porque no está. Hallar es observar lo que hay. Elegir es razonar lo que debería haber. Por eso una elección nunca puede llevar la etiqueta de hallazgo. No viene del mismo sitio.

Esto convierte la frontera en una decisión de arquitectura, no de comodidad. No dices el agente hace hasta aquí porque más allá se equivoca. Dices el agente halla y el arquitecto elige porque mezclar las dos cosas destruye la fiabilidad del modelo. Es una frontera epistémica, sobre qué clase de conocimiento es cada cosa, y resulta que coincide exactamente con la frontera de quién hace qué. Lo recuperado, que tiene fuente, lo puede traer el agente. Lo decidido, que no tiene más fuente que el juicio, lo tiene que poner el arquitecto, y tiene que quedar marcado como tal.

Conviene insistir en una asimetría que la gente subestima. El agente puede proponer opciones de diseño, y debe hacerlo, porque ahí es muy útil. Lo correcto es que el agente, tras recuperar el dominio, presente al arquitecto un abanico de arquitecturas posibles, cada una con su evidencia y con sus consecuencias, especialmente sus consecuencias sobre la verificación posterior. Esta opción es más barata de probar pero consolida dos reglas en una. Esta otra es más fiel pero más cara de verificar. El agente es excelente preparando esa mesa de opciones. Lo que no puede hacer es sentarse a la mesa y elegir por ti, porque elegir es comprometer el juicio, y el juicio responde ante el negocio, no ante una herramienta. El agente sirve el menú. El arquitecto elige.

Queda una pregunta práctica. Si lo recuperado y lo decidido tienen que distinguirse, cómo se distinguen en la práctica, dentro del modelo, de forma que cualquiera que lo lea sepa qué está mirando. La respuesta es una disciplina sencilla y rigurosa, y es el siguiente capítulo.

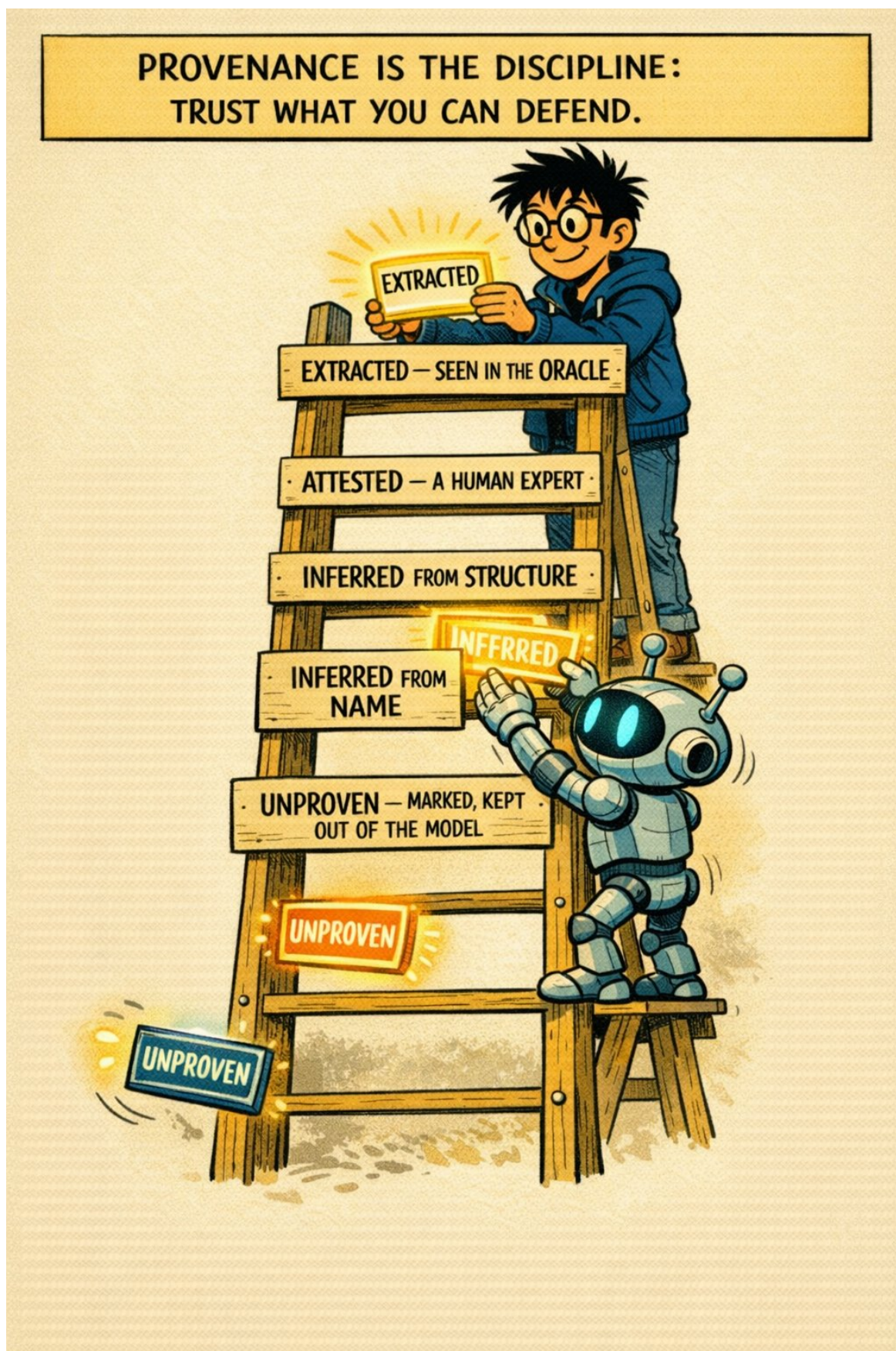
Capítulo 7

La disciplina de procedencia

Un modelo de dominio recuperado de un sistema viejo es una mezcla de cosas de orígenes muy distintos. Algunas las has extraído directamente del comportamiento observado. Otras las has inferido de la estructura, parecen ciertas pero no las has confirmado. Otras te las ha confirmado un experto del negocio que lleva treinta años con el sistema. Otras son decisiones de diseño que has tomado tú. Y otras son simplemente conjeturas que aún no has probado. Todas conviven en el mismo modelo, y desde fuera parecen iguales, frases que describen el dominio. Esa apariencia de igualdad es un peligro, porque te invita a confiar en todas por igual, cuando merecen confianzas muy distintas.

La solución es una disciplina que pido que tomes en serio aunque suene burocrática, porque es lo que sostiene la fiabilidad de todo el método. **Ninguna afirmación entra en el modelo sin declarar su procedencia.** Cada regla, cada frontera, cada invariante que escribes lleva una marca que dice de dónde viene y, por tanto, cuánto puedes fiarte de ella.

Las categorías de procedencia son pocas y claras. Hay afirmaciones extraídas, observadas directamente en el comportamiento del oráculo, las más fiables, porque las has visto pasar. Hay afirmaciones inferidas de la estructura, deducidas de cómo está organizado el sistema, plausibles pero no confirmadas, porque la estructura puede engañar. Hay afirmaciones inferidas del nombre, sugeridas por cómo se llama algo, las más débiles, porque los nombres en sistemas viejos mienten con frecuencia. Hay afirmaciones atestiguadas por un experto, que un humano que conoce el negocio confirma. Y hay afirmaciones sin probar, conjeturas marcadas honestamente como tales para que nadie las tome por hechos. Las marcas concretas que uses dan igual, importan las distinciones que capturan. La más delicada de todas es la que separa un comportamiento extraído, que viste pasar, de su interpretación, que quizá nadie ha confirmado: lo primero es un hecho del sistema, lo segundo no entra en el modelo hasta que el negocio lo resuelve.



CardDemo • paso 3/11

El control de crédito vive en CBTRN02C, párrafo 1500-VALIDATE-TRAN. Mira la exposición del ciclo, no ACCT-CURR-BAL:

```

COMPUTE WS-TEMP-BAL = ACCT-CURR-CYC-CREDIT
                    - ACCT-CURR-CYC-DEBIT
                    + DALYTRAN-AMT
IF ACCT-CREDIT-LIMIT >= WS-TEMP-BAL
  CONTINUE
ELSE
  MOVE 'OVERLIMIT TRANSACTION'
  TO WS-VALIDATION-FAIL-REASON-DESC
END-IF

```

Ese comportamiento es extraño: lo viste pasar en el oráculo. Si es una regla deliberada o un defecto de hace décadas es, en cambio, una afirmación sin probar. El método no la decide: la lleva al registro de decisiones para que la resuelva el negocio.

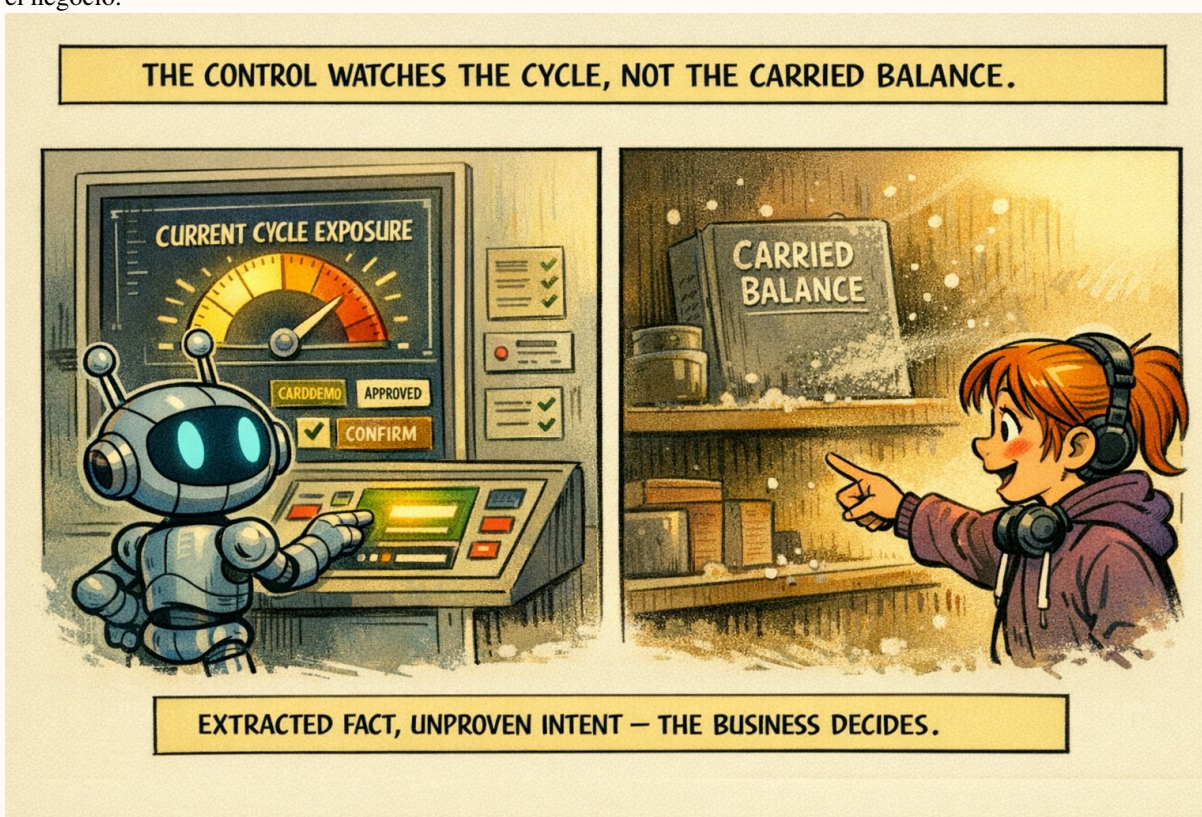


Figura. La rareza del crédito sobre una línea de tiempo de dos ciclos de facturación: el control mira la exposición del ciclo en curso e ignora el saldo arrastrado del ciclo anterior.

Hay un par de sutilezas que merecen detenerse, porque es donde la disciplina demuestra su valor. La primera. Lo atestiguado por un experto no es lo mismo que lo extraído del oráculo, aunque los dos parezcan firmes. El experto puede recordar mal, puede describir cómo cree que funciona el sistema en vez de cómo funciona, puede estar describiendo una versión de hace diez años. Su testimonio es valioso, pero es testimonio, no observación. Por eso lleva su propia marca, distinta de lo extraído. Cuando el testimonio del experto y la observación del oráculo discrepan, la disciplina de procedencia te avisa de que tienes un conflicto que resolver, en vez de dejarte elegir a ciegas. Y casi siempre gana el oráculo, porque el sistema hace lo que hace, diga lo que diga el recuerdo.

Conviene además distinguir dos expertos que la palabra esconde. El experto del negocio habita el espacio del problema: sabe por qué existe la regla y qué pasaría si cambiara. El veterano del mainframe, el que lleva veinte años manteniendo

el sistema, habita el espacio de la solución: sabe qué giro del código es un idioma de la plataforma y cuál esconde una decisión, dónde están enterrados los casos que muerden, y si la réplica que hace de oráculo se comporta como el original donde más importa. Su testimonio es el canal atestiguado para desambiguar mecanismo de lógica, justo la frontera donde el capítulo cuatro situó los errores más caros. Los agentes no lo desplazan: lo ascienden. Deja de gastar sus años en el mantenimiento trivial y pasa a hacer lo que solo él puede hacer, interpretar al oráculo.

La segunda sutileza conecta este capítulo con el anterior y cierra la idea de la frontera. Hay una regla de oro. **Lo que no puede citar ninguna fuente no es un hallazgo, es una elección, y debe salir del modelo de lo recuperado hacia un registro de decisiones.** Si escribes una afirmación sobre el dominio y al intentar marcar su procedencia descubres que no puedes señalar de dónde viene, que no la extrajiste ni la inferiste ni te la atestiguó nadie, entonces no estás describiendo el sistema viejo. Estás decidiendo algo sobre el nuevo. Esa afirmación no pertenece al modelo de lo recuperado. Pertenecer al registro de decisiones de diseño, que es harina de otro costal, el insumo del rediseño, no parte del retrato del dominio.

Esta regla es exactamente la frontera del capítulo anterior, hallar frente a elegir, hecha operativa. La procedencia es el mecanismo por el que la frontera entre el agente y el arquitecto deja de ser una intención y se vuelve una práctica que puedes auditar. Cualquiera puede coger el modelo y preguntar, de cada afirmación, de dónde viene. Si todo lo recuperado tiene fuente y todo lo que no tiene fuente está en el registro de decisiones, la frontera está limpia. Si encuentras afirmaciones sin fuente camufladas entre las recuperadas, sabes que alguien, un humano o un agente, ha colado una elección disfrazada de hallazgo, y sabes que el modelo está contaminado justo ahí.

Hay quien dirá que esto es demasiada ceremonia para un trabajo de ingeniería. La respuesta es que el coste de la ceremonia es pequeño y el coste de no tenerla es enorme y diferido. Sin disciplina de procedencia, seis meses después nadie recuerda si la regla del caso límite la observasteis en el sistema viejo o la inventó alguien en una reunión, y esa diferencia es la diferencia entre un hecho del negocio y una opinión que quizá esté mal. La procedencia es barata de mantener si la mantienes desde el principio, y carísima de reconstruir si no lo hiciste. Es, en el fondo, la misma lección que la verificación que veremos más adelante. Más vale demostrar que asumir, y más vale saber de dónde viene cada cosa que confiar en que alguien se acuerda.

Con la frontera dibujada y la procedencia en su sitio, tenemos las dos mitades del método agéntico. Pero una frontera que hay que recordar y una disciplina que hay que querer cumplir son frágiles. Lo que las sostiene cuando nadie las vigila no es la buena voluntad, es una máquina. Y de esa máquina va el capítulo que cierra esta parte.

Capítulo 8

El almacén

Los dos capítulos anteriores trazaron una línea y pidieron una disciplina. La línea separa lo que el agente halla de lo que el arquitecto elige. La disciplina exige que cada afirmación declare su procedencia. Las dos son correctas, y las dos son frágiles, porque dependen de que alguien se acuerde de respetarlas. Una línea que solo existe en la cabeza de quien la trazó se cruza el primer día que aprieta el calendario. Una disciplina que solo se sostiene por buena voluntad se abandona en la primera prisa.

La pregunta de este capítulo es qué hace que la línea y la disciplina se cumplan cuando nadie las vigila. **La respuesta es que no las sostiene la voluntad, las sostiene una máquina.** Llamo almacén a esa máquina. No es una herramienta concreta. Es una forma de organizar el trabajo del agente para que la frontera y la procedencia dejen de ser intenciones y pasen a ser propiedades del sistema, comprobables y difíciles de saltarse.

El substrato determinista

En el capítulo seis quedó dicho que el agente no razona bien sobre código crudo, sino sobre proyecciones, mapas que destilan una dimensión del sistema. Quedó una pregunta sin responder, y es la más importante de todas. Si el agente razona sobre la proyección, quién construye la proyección, y por qué fiarse de ella.

Hay una respuesta tentadora y equivocada: que la construya el agente, leyendo el sistema y dibujando el mapa. Es equivocada porque entonces la proyección heredaría la falibilidad del agente. Un agente que dibuja el mapa que luego va a interpretar puede trazar mal una arista, inventar una llamada que no existe, omitir una que sí, y después razonar con total seguridad sobre un mapa que miente. El error más caro no es el agente que se equivoca al leer un mapa. Es el agente que se equivoca al dibujarlo y nadie lo nota.

Por eso la proyección no la inventa el agente. **La proyección se deriva mecánicamente del sistema, con un proceso determinista que no opina.** Se analizan el código y los datos con reglas fijas, y se vuelca el resultado en una estructura navegable: qué llama a qué, qué dato fluye hacia qué decisión, qué condición protege qué estado. Ese volcado es reproducible. Si lo corres dos veces sobre el mismo sistema, obtienes el mismo mapa. Y cada elemento del mapa puede señalar de qué parte del sistema salió, que es la procedencia del capítulo anterior aplicada al substrato: cada arista cita su origen en el código.

Sobre ese substrato determinista, el agente hace lo que hace bien. Busca, relaciona, propone. Una búsqueda por significado encuentra candidatos: esta estructura se parece a un agregado, esta condición se parece a un invariante. Y el grafo determinista verifica si esos candidatos se sostienen, si la llamada existe de verdad, si el dato fluye como se cree. La búsqueda propone, el grafo dispone. El agente nunca verifica contra su propia memoria, que inventa con aplomo. Verifica contra el substrato, que no inventa.

En la práctica. El substrato suele materializarse como un grafo de propiedades consultado junto a un índice semántico. Ese emparejamiento, búsqueda semántica que propone y grafo que verifica, es lo que la industria llama GraphRAG. La máquina de estados puede ser una librería ligera de estados y transiciones. A los agentes

de cada estado los coordina un framework de orquestación. Los nombres concretos cambiarán. Lo que no cambia es la idea: substrato determinista, estados con acceso acotado, guardas que no se saltan.

La máquina de estados

El substrato hace fiable lo que el agente lee. Falta hacer fiable lo que el agente hace. Recuperar un dominio no es un solo acto, es una secuencia: leer, proponer, confrontar con el oráculo, ratificar, decidir arquitectura, verificar. Si esa secuencia se deja al criterio del momento, se desordena. Se ratifica antes de confrontar. Se decide arquitectura sobre un modelo a medio recuperar. Se verifica lo que aún no está decidido.

La solución no es una idea nueva. Es la forma más vieja que tiene la informática de ser fiable: una máquina de estados. La máquina de Turing, el modelo abstracto con el que arranca la teoría de la computación, es exactamente eso, una máquina cuyo siguiente paso queda determinado por su estado y su entrada, sin margen para la sorpresa. Y es, punto por punto, lo contrario de un agente, que ante la misma entrada puede responder cosas distintas. El armazón vive de ese contraste: pone lo impredecible, el agente, a trabajar dentro de lo predecible, la máquina.

El armazón organiza la secuencia como esa máquina de estados. **Cada estado tiene una sola meta, el agente que la persigue, el contexto que necesita y nada más, y un conjunto acotado de herramientas a las que puede llegar.** Un estado para recuperar la táctica. Otro para confrontar candidatos con el oráculo. Otro para ratificar. Otro para proponer arquitectura. El agente que recupera no alcanza las herramientas que deciden, porque en ese estado no decide. El que verifica no puede tocar el modelo, porque en ese estado no modela. El contexto de cada estado se limpia al entrar, para que el ruido de una tarea no contamine la siguiente.

Esto no es burocracia. Es lo que impide que las tareas se mezclen. Un agente con acceso a todo y memoria de todo tiende a hacer de todo a la vez, y hacer de todo a la vez es exactamente cómo una elección se disfraza de hallazgo y una conjetura se cuela como hecho. La máquina de estados es la frontera del capítulo seis hecha topología: el agente no puede cruzar de hallar a elegir porque son estados distintos, con accesos distintos, y el paso entre ellos está guardado.

Las guardas son los gates del libro

Entre estado y estado hay una guarda. Una guarda es una condición determinista que debe cumplirse para que la máquina avance. No es un consejo ni una lista que alguien repasa. Es un cerrojo. Si la condición no se cumple, la transición no ocurre, y no hay forma de empujarla a mano sin dejar rastro.

Aquí el armazón recoge piezas que en otros capítulos aparecen sueltas. **Lo que el libro llama gates son guardas de esta máquina, no listas de buenas intenciones al final de una rama.** La ratificación es una guarda: solo pasa al modelo lo que puede defender su procedencia, y lo dudoso no transiciona, se queda o vuelve a confrontarse. La ley de orden, que veremos en la Parte IV, es una guarda: no se entra en el estado de refactor de calidad hasta que la guarda de equivalencia está en verde. El cutover, en la Parte V, es una guarda: no se corta hasta que se cumplen sus condiciones, equivalencia, cobertura, sombra y divergencias resueltas.

Cuatro disciplinas que parecían independientes resultan ser la misma cosa vista en sitios distintos: condiciones de transición de una máquina que no deja avanzar hasta que se cumplen. Esa es la diferencia entre un método que te pide hacer las cosas en orden y un método en el que el desorden es, sencillamente, imposible.

Señales y observabilidad

Una guarda solo es tan buena como la información con la que decide. Si la guarda de equivalencia pregunta «coinciden viejo y nuevo» y la respuesta es un sí o un no sin matices, no sirve. Necesita saber sobre qué coinciden, con cuánta cobertura, qué casos se probaron y cuáles no. Esa información son las señales: lo que el sistema emite sobre sí mismo mientras trabaja.

La observabilidad no es un lujo operativo que se añade al final. Es lo que permite que las guardas decidan con fundamento y que la frontera sea auditable. **Cada decisión de la máquina deja un rastro con su procedencia, de modo que cualquiera puede preguntar después por qué se avanzó, y la máquina lo responde.** Por qué se ratificó esta regla. Con qué evidencia se cerró esta equivalencia. Qué cobertura tenía la especificación cuando se permitió el corte.

La cobertura merece una palabra, porque es la que pone número al residuo honesto. En el capítulo del oráculo quedó dicho que el sistema viejo solo sabe lo que se le pregunta, y que lo que no se ejercita queda sin capturar. La cobertura es la medida de cuánto se ha ejercitado. No elimina el residuo, ningún proceso captura lo que nunca se manifestó, pero lo acota y lo hace visible. Una guarda que se abre con cobertura pobre no certifica equivalencia, certifica que coincidieron donde miraste. La observabilidad es lo que impide confundir las dos cosas.

Quién gobierna la máquina

Queda decir dónde está el humano. La tentación, cuando la máquina funciona, es ponerlo dentro del bucle, aprobando cada paso, o sacarlo del todo y dejar que corra sola. Las dos son errores.

El arquitecto no está dentro del bucle, está sobre el bucle. No aprueba cada transición, eso lo hacen las guardas deterministas, que para eso están y lo hacen mejor que una persona cansada un viernes. El arquitecto vigila la máquina, lee sus rastros, y decide en los puntos donde la máquina, por diseño, no decide: dónde está la frontera, qué arquitectura, qué hacer con lo que no tiene procedencia. **El agente opera dentro de la máquina. El arquitecto gobierna la máquina. No son el mismo trabajo y no se delegan el uno en el otro.**

Esto cierra el argumento de la Parte II. El agente halla, y la máquina garantiza que solo lo que tiene fuente entra como hallazgo. El arquitecto elige, y la máquina garantiza que la elección queda marcada como elección y no se disfraza de otra cosa. La frontera ya no es una intención que hay que recordar. Es la forma de la máquina.

Con la frontera trazada, la procedencia en su sitio y el almacén haciéndolas cumplir, las dos mitades del método agéntico encajan: el agente que halla y el arquitecto que elige, separados por una máquina que no deja que se confundan. Tenemos el cómo. Falta una pregunta de actitud, la que decide cuándo conviene usar esto en serio y cuándo no, y por qué un método joven que se trata como maduro fracasa. Ahí entra Beck.

Parte III

Explorar antes de productizar

THE DOMAIN ONTOLOGY

— THE LANGUAGE YOU VERIFY IN —

1 The first move in modernisation is not choosing a target architecture. It is recovering the domain, stripped of the mechanics four decades have wrapped around it.

2 What you recover is the ubiquitous language of Domain-Driven Design: the single vocabulary business and engineering both speak, with no translation layer in between.

3 It does two jobs.

1. UNDERSTANDING & DESIGN
It gives the people who never read a line of COBOL a shared, precise understanding of what the business does, and a way to design the future in business terms instead of file formats.

2. VERIFICATION
And then it does the job almost no one asks of it. It becomes the language you verify in.

4 Watch the legacy run, in production or in the oracle (recompiled to run as a behavioural reference you can replay), and you capture behaviour. Inputs in, outputs out.

5 Translate that behaviour into a domain rule a business person would recognise and sign off, and both systems can be held to it.

6 A transaction would push an account past its credit limit, and the legacy declines it.

7 Agents earn their place in two roles.

8 VERIFICATION: PROPOSE

THE ONTOLOGY IS A SUBSTRATE IN ITS OWN RIGHT.
One ontology can be realised by an infinity of target solutions. The architecture is free to change. The meaning is not. Fidelity is owed to the invariant, not to the structure.

THE ONTOLOGY (THE DOMAIN)

ACCOUNT CUSTOMER TRANSACTION
CREDIT LIMIT PAYMENT DECLINE ...

COBOL MAINFRAME COMPILED COBOL MODULAR MONOLITH MICROSERVICES EVENT-DRIVEN SYSTEM AGENTIC SYSTEM

LEGACY (TODAY)
Account: A123
Limit: \$1,000
Txn: \$1,200
DECLINED

THE RULE (DOMAIN)
A TRANSACTION THAT BREACHES THE CREDIT LIMIT IS DECLINED.

NEW SYSTEM (TOMORROW)
Account: A123
Limit: \$1,000
Txn: \$1,200
DECLINED

DISCOVERY: PROPOSE
An agent reads what no human reads end to end and proposes the domain rules buried in the mechanics.

CANDIDATE RULE
A TRANSACTION THAT BREACHES THE CREDIT LIMIT IS DECLINED.

RATIFIED

EXECUTABLE SPECIFICATIONS (BDD)
Runs against Legacy (Oracle) PASS
Runs against New System PASS

ONE ONTOLOGY. INFINITE IMPLEMENTATIONS. ARCHITECTURE IS FREE TO CHANGE. MEANING IS NOT.

COBOL MAINFRAME COMPILED COBOL MODULAR MONOLITH MICROSERVICES EVENT-DRIVEN SYSTEM AGENTIC SYSTEM

Capítulo 9

El 3X aplicado a la modernización

Kent Beck observó que el software vive tres fases muy distintas, y que confundirlas es una de las causas más comunes de desastre. Las llamó Explore, Expand, Extract, las tres equis. En Explore buscas qué funciona, y la verdad incómoda de esta fase es que la mayoría de lo que pruebas no funciona, así que el valor está en probar barato, aprender rápido y descartar sin pena. En Expand algo ha funcionado y de repente tienes que crecer para atender una demanda que se dispara, y el problema deja de ser qué hacer y pasa a ser aguantar. En Extract ya sabes qué haces y lo haces a escala estable, así que el juego es exprimir eficiencia, pulir, optimizar. Cada fase pide una mentalidad distinta, y aplicar la mentalidad de una a otra es el error. Optimizar en Explore es pulir algo que vas a tirar. Explorar en Extract es introducir caos donde hacía falta estabilidad.

La tesis de este capítulo, y una de las decisiones de actitud más importantes del libro, es que **una metodología de modernización joven está en Explore, y tratarla como si estuviera en Extract es el camino más rápido al fracaso**. Esto se aplica en dos niveles, y conviene separarlos porque es fácil mezclarlos.

El primer nivel es el método mismo. Este libro describe un método que está en su fase Explore. No ha sido validado contra cientos de migraciones de producción. Es una hipótesis razonada, un argumento de por qué debería funcionar. Tratarlo como un proceso maduro, codificarlo en una metodología corporativa rígida y desplegarlo a escala antes de haberlo recorrido en pequeño, sería cometer el error de Beck en su forma más pura, productizar algo que todavía estás explorando. La actitud correcta es la contraria. Recorrer el método en ciclos pequeños y rápidos, sobre piezas acotadas, aprendiendo de cada ciclo, descartando lo que no funcione, hasta que la evidencia justifique pasarlo a Expand. El método se gana el derecho a escalar, no se lo arroga.

El segundo nivel es cada encargo concreto de modernización, sea un proyecto acotado o una transformación de empresa de varios años. También empieza en Explore, aunque la presión organizativa empuje a fingir que empieza en Extract. La presión dice, ya sabemos lo que hay que hacer, planifiquemos las dieciocho fases y ejecutemos. Es mentira, y es una mentira cara. Al principio de un encargo no sabes lo bastante, y conviene admitir por qué. No eres experto en el negocio del cliente. Eres experto en modernizar, que es otro oficio. No conoces el dominio real escondido en el sistema, no sabes dónde están las fronteras naturales, no sabes qué partes son lógica y cuáles mecanismo, no sabes dónde están los casos raros que te van a morder. Por eso el dominio no se trae de la cabeza del que moderniza. Se recupera de dos fuentes: del sistema, que muestra lo que hace, y de los expertos del negocio, que no tienen por qué ser ingenieros y que ponen el nombre y el porqué a lo que el sistema solo muestra hacer. Fingir un plan detallado sobre ese desconocimiento produce un plan de ficción que la realidad destroza en el primer mes.

La alternativa es empezar por lo que merece llamarse un spike de arquitectura, tomando prestado el término de la práctica ágil. Un spike es un experimento acotado en el tiempo cuyo objetivo no es producir entregable sino producir conocimiento. Coges una porción del sistema fina pero completa, idealmente una que toque las partes más temidas. Fina importa. No es una capa horizontal, no es solo recuperar o solo verificar. Es una aguja que atraviesa todos los pisos del ascensor a la vez, de la sala de máquinas al penthouse: recuperación, rediseño y verificación sobre una franja estrecha del dominio. Lo cruza todo, y poco de cada cosa. No para entregarla, sino para aprender. Al final del spike sabes cosas que ningún plan previo podía darte. Sabes cuánto cuesta de verdad recuperar el dominio de este sistema. Sabes si tu oráculo es fiel. Sabes dónde están los casos raros. Sabes qué fronteras tienen sentido. Con ese conocimiento, el plan que escribes después es

un plan de verdad, no de ficción.

Aquí es donde el abaratamiento agéntico del capítulo cinco cambia las reglas de un modo profundo. En un mundo sin agentes, los spikes son caros, porque recuperar dominio a mano es lento, así que la tentación de saltárselos y planificar a ciegas es fuerte, y muchas veces gana. En un mundo con agentes, el spike es barato, porque la recuperación que antes costaba semanas cuesta días. Y cuando el spike es barato, puedes permitirte varios, explorar varias porciones, varias hipótesis de fronteras, varias arquitecturas candidatas, antes de comprometerte con ninguna. Lo que antes era un lujo que casi nadie se permitía, explorar de verdad antes de decidir, se vuelve la opción por defecto. El agente no solo acelera el trabajo. Hace asequible la prudencia.

Hay una disciplina que acompaña al 3X y que conviene declarar sin rodeos. No productices prematuramente. La tentación, cuando un spike sale bien, es declarar victoria, congelar el enfoque y aplicarlo en masa. Es la tentación de saltar de Explore a Extract sin pasar por Expand, de convertir un experimento exitoso en un estándar antes de haberlo probado en varios casos distintos. Un solo spike exitoso no prueba que el método funcione, prueba que funcionó una vez, que es muchísimo menos. La señal de que estás listo para Expand no es que algo funcionara, es que entiendes por qué funcionó lo bastante bien como para predecir cuándo no funcionará. Hasta tener eso, sigues en Explore, y está bien estar en Explore. Es donde se aprende.

Esta actitud, recorrer en ciclos rápidos antes de comprometerse, encaja con el ascensor del capítulo cinco y con la filosofía ágil original, la de verdad, la de antes de que la palabra se llenara de ceremonias: ciclos cortos, aprender del mundo real, descartar sin pena lo que no funciona. Cada ciclo es un viaje completo de ascensor, bajas a recuperar, subes a modelar el dominio, paras en la planta intermedia a decidir la arquitectura, bajas a verificar. La diferencia entre hacerlo bien y mal no está en las plantas, que son las mismas, sino en la velocidad y la humildad del viaje. Rápido, para aprender mucho por unidad de tiempo. Humilde, para tirar sin pena lo que el viaje revele que no sirve. Con esa actitud puesta, el primer viaje de verdad es bajar a la sala de máquinas a recuperar el dominio. Ese es el siguiente capítulo.

Capítulo 10

Recuperar el dominio

Recuperar el dominio es el acto central de la modernización. Es el cimiento del que cuelga todo lo demás, y un cimiento torcido no se endereza más arriba. El objetivo es producir un retrato fiel del dominio que el sistema viejo encarna, expresado en los términos del Diseño Dirigido por el Dominio, despojado del mecanismo, y con cada afirmación marcada por su procedencia. Llamemos a ese retrato el modelo recuperado. No es el sistema viejo y no es el nuevo. Es el dominio, el qué, separado del cómo viejo y todavía sin comprometerse con ningún cómo nuevo.

Conviene ser muy preciso sobre qué es y qué no es este modelo, porque la confusión aquí envenena todo lo demás. El modelo recuperado describe el estado actual del dominio, no el deseado. Es un retrato del as-is, no del to-be. Esto sorprende a quien espera que modernizar empiece por dibujar el sistema ideal. No. Empieza por entender con precisión lo que hay, porque no puedes preservar la semántica de algo que no has entendido, y preservar la semántica es el contrato de la fidelidad del capítulo cuatro. El sistema ideal viene después, y viene como una decisión de diseño, marcada como tal, no como parte del retrato.

El modelo recuperado tiene dos capas, las dos clásicas del DDD. La capa táctica describe el interior de cada parte, las entidades y sus identidades, los agregados y los invariantes que protegen, los objetos de valor, las relaciones. La capa estratégica describe el conjunto, el mapa de contextos, las fronteras entre las distintas partes del dominio y cómo se relacionan. Recuperar las dos es el trabajo. La táctica te dice qué reglas viven dentro de cada concepto. La estratégica te dice dónde están las costuras del dominio, que es donde más adelante podrás cortar.

Aquí hay un matiz que mucha gente pasa por alto y que conviene clavar, porque parece una contradicción y no lo es. Si lo que recuperas es el dominio, el qué, por qué el modelo recuperado contiene también estructura, que es del cómo. Porque un retrato fiel no puede ignorar cómo está organizado el sistema hoy. El modelo recuperado es un mapa nuevo del negocio, y en ese mapa anotas dos cosas distintas, con estatus distinto.

La primera es la capa obligatoria: los invariantes y la semántica, el espacio del problema. Es el qué, sobrevive a cualquier rediseño, y es sagrada. La segunda es una capa informativa: la estructura tal como el sistema corre y se observa hoy, las fronteras que tiene puestas, qué módulo guarda qué, dónde están las costuras. Es espacio de la solución, pero no de un diseño ideal ni del que alguien imaginó hace décadas, sino del sistema vivo tal como el oráculo lo muestra. La recuperas porque es información valiosa sobre cómo el sistema entiende hoy su propio dominio, no porque sea sagrada. Es, de hecho, justo lo que vienes a cambiar.

Por eso no hay contradicción. Recuperas el dominio, y de paso anotas la estructura actual como dato, marcada como re-decidible. Confundir las dos capas es el error. Si tratas la estructura recuperada como sagrada, vuelves a la trampa de la traducción. Si tratas los invariantes como negociables, rompes el sistema. El mapa lo dibuja todo, pero te dice de cada trazo a qué capa pertenece.

El proceso de recuperación, en el método agéntico, reparte el trabajo según la frontera del capítulo seis. El agente halla. Recorre el sistema, o mejor, recorre las proyecciones del sistema, los mapas que destilan sus dimensiones, y extrae candidatos, esta estructura parece un agregado, esta condición parece un invariante, este dato parece la identidad de esta entidad. Cada candidato viene con su procedencia, lo extraje del comportamiento, lo inferí de la estructura, lo sugiere el nombre. El arquitecto ratifica. Mira los candidatos, los confronta con el oráculo cuando hay duda, los confirma o los

rechaza, y sobre todo decide qué hacer con lo que no tiene fuente clara.

CardDemo • paso 4/11

El agente no lee los treinta y un programas en crudo. Razona sobre una proyección: un grafo donde cada programa y cada fichero es un nodo, y cada acceso es una arista. Sobre esa proyección, el hotspot no hay que buscarlo, emerge solo. Una consulta lo saca a la luz:

```
MATCH (p:Program)-[:WRITES_FILE]->(f:File {name: 'ACCTFILE'})
RETURN p.name AS programa
ORDER BY programa
```

La respuesta son cuatro programas escribiendo el mismo fichero. Eso no estaba en ninguna documentación. Estaba en el comportamiento, y el grafo lo hizo visible. Es la primera prueba concreta de que el agente razona mejor sobre el mapa que sobre el territorio.

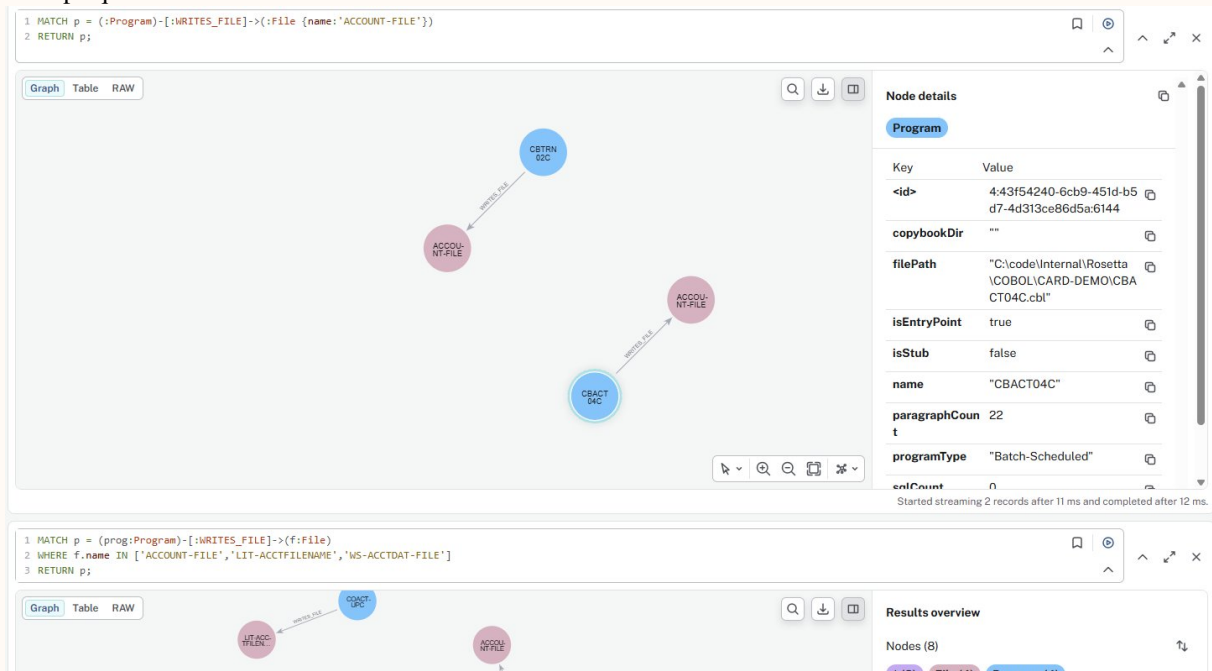


Figura. El grafo Atlas en Neo4j: la consulta `WRITES_FILE` sobre el fichero de cuentas, con los programas que lo escriben representados como nodos del grafo.

Hay una idea sobre la ratificación que merece destacarse porque corrige un malentendido habitual. La ratificación no es donde se resuelve la incertidumbre. Es donde se prohíbe que la incertidumbre entre. Suena igual pero no lo es. Resolver la incertidumbre suena a que en la reunión de ratificación se discuten las dudas y se llega a un acuerdo sobre lo dudoso. Mal. Lo dudoso no se ratifica, se marca como dudoso y se manda a verificar contra el oráculo o a investigar. Lo que se ratifica es lo que ya está sólido, lo que tiene fuente firme. La ratificación es un filtro que deja pasar solo lo que puede defender su procedencia, no un tribunal que convierte conjeturas en hechos por consenso. Si sales de la ratificación habiendo bendecido como cierto algo que entró como dudoso, sin nueva evidencia, has corrompido el modelo con la autoridad de una firma.

Una advertencia honesta para cerrar. La recuperación nunca es completa. Siempre quedará comportamiento en rutas que no se ejercitan, casos que el oráculo no ha visto porque no se le preguntaron, reglas dormidas que solo despiertan en circunstancias raras. Eso es residuo irreducible, y la actitud correcta no es fingir que no existe sino marcarlo. El modelo recuperado honesto incluye sus propios huecos, las zonas donde sabe que no sabe. Esos huecos no son un fracaso de la recuperación, son su parte más valiosa, porque te dicen exactamente dónde el sistema nuevo te puede sorprender, y por tanto dónde vigilar. Un modelo que pretende ser completo miente. Uno que marca sus huecos te dice dónde mirar.

Con el dominio recuperado, retratado, marcado y con sus huecos señalados, el ascensor sube. Pero antes de decidir

ninguna arquitectura, hay que decidir qué merece una. Eso es triar, y es el siguiente capítulo.

Capítulo 11

Recuperar y luego triar

Una vez recuperado el dominio, cuando por fin ves lo que tienes, la estrategia deja de ser una talla única y se vuelve un triaje, en términos llanos de Diseño Dirigido por el Dominio. **El dominio recuperado no es una lista de reescritura. Es el mapa con el que decides qué reescribir, qué encapsular y qué jubilar.** El mapa que acabas de recuperar ya trae la respuesta dibujada: el color es el triaje.

CardDemo • paso 5/11

Elevado el grafo a mapa de contextos, esto es lo que la recuperación devuelve de CardDemo. No es un diseño, es un retrato: la estructura tal como el sistema vivo la corre hoy. El núcleo, servicing de cuentas, posting de movimientos y servicing de tarjetas, en naranja; el soporte, donde caen customer, interés, consulta, facturación e informes, en azul; identidad y acceso, el único genérico, en gris. Y en rojo el hotspot, el fichero de cuentas reescrito por cuatro contextos a la vez. El color ya es el triaje: naranja se reescribe, azul se encapsula, gris se compra.

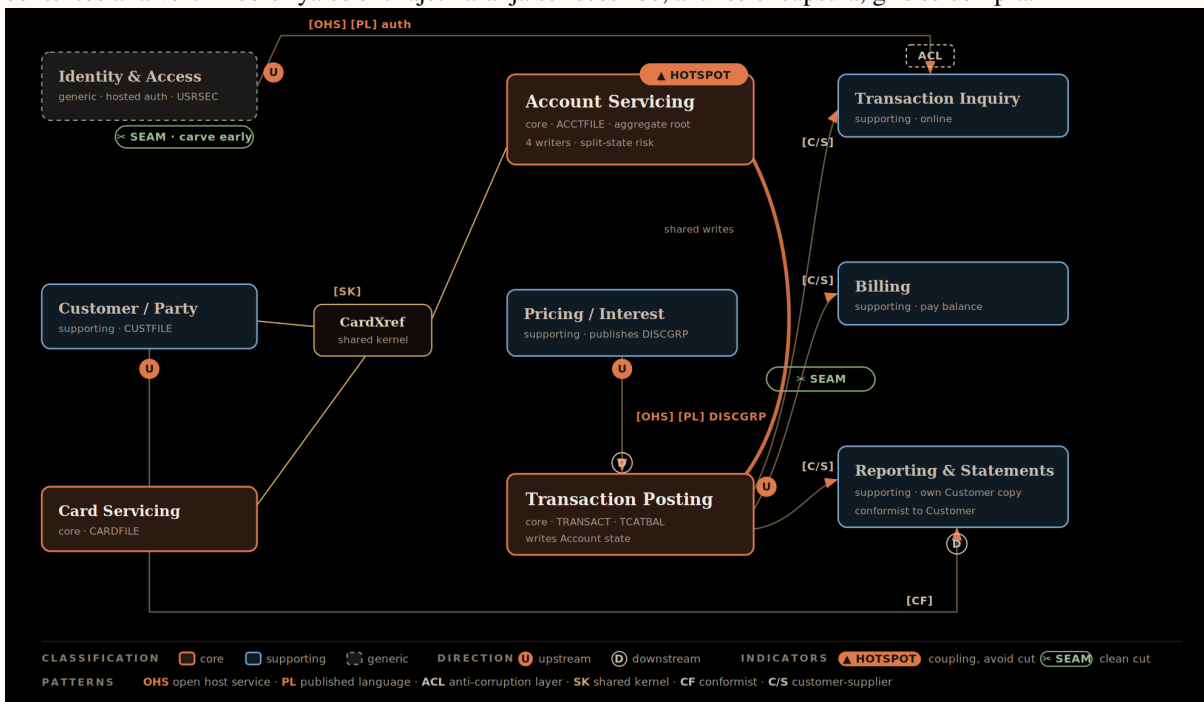


Figura. El mapa de contextos as-is de CardDemo tal como lo devuelve la recuperación: núcleo en naranja, soporte en azul, genérico en gris, y el hotspot de escritura compartida sobre ACCTFILE en rojo.

Los subdominios genéricos, las commodities del negocio, van a solución de mercado. No hay gloria en mantener código

a medida para problemas que el mercado resolvió hace años, y cada commodity que retiras libera a gente escasa para el trabajo que sí importa.

Los subdominios de soporte, y todo código que sea estable y simple, se encapsulan y se dejan en paz. Estas cargas dejan de consumir a tus expertos sénior en COBOL, que hacen falta donde solo ellos sirven, atestiguar, interpretar los hotspots, validar la fidelidad de la réplica. Con el conocimiento de dominio recuperado y el oráculo en su sitio, un agente o un desarrollador júnior pueden hacer cambios menores y bien guardados con seguridad: añadir un campo a un copybook no debería requerir un héroe. Y lo que hace seguro ese trabajo no es la fe en el agente: es la especificación capturada en el contrato del servicio, la red de regresión de la Parte IV aplicada a lo encapsulado.

Aquí va una postura que rara vez se oye en un artículo sobre IA: conserva el COBOL. El COBOL que es estable, correcto y hace su trabajo en silencio es un activo, no una emergencia. La jugada madura es encapsularlo tras un contrato explícito, un servicio de host abierto con un lenguaje publicado, para que el resto del parque consuma capacidades en vez de interioridades. Si la plataforma misma debe moverse, reubica el COBOL como COBOL, recompilado sobre un runtime moderno y contenerizado, tras ese mismo contrato. Mejor aún, cuando la economía lo permita, mantenlo donde está y exponlo igual, por un extremo remoto sobre HTTP. En cualquier caso el principio es idéntico: **envolver para invocar, aislar para no heredar. Encapsular compra opcionalidad; transpilar la gasta.**

La comparación con la transpilación merece hacerse explícita, porque las dos se venden como salidas del mainframe y no se parecen en nada. Recompilar conserva el artefacto que se mantiene: el COBOL sigue siendo COBOL, el equipo que lo conoce sigue siendo competente, y el compilador garantiza por construcción la semántica de los datos, el decimal empaquetado, la codificación, la ordenación, la precisión, que es justo donde una conversión rompe en silencio. Transpilar sustituye el artefacto por código generado que nadie escribió y nadie quiere mantener, el veredicto del cementerio. Por eso la misma técnica que en el capítulo tres construía un oráculo fiable es aquí un destino digno: preserva la semántica por diseño en vez de prometerla.

En la práctica. Un compilador como Raincode recompila el COBOL a IL de .NET y lo ejecuta sobre el runtime moderno, preservando por diseño la representación de los datos, el COMP-3, el EBCDIC, la ordenación y la precisión decimal, de modo que el programa que corre contenerizado sigue siendo, a efectos de mantenimiento, el mismo COBOL. El compilador es la instancia; lo durable es el criterio: para lo que no vas a rediseñar, elige la vía que conserva el artefacto y la semántica, no la que los sustituye.

Esto no contradice la trampa de la relocalización del principio del libro; la completa. Allí el error era confundir reubicar con modernizar el núcleo, vender un cambio de plataforma como si fuera una transformación del dominio. Aquí, para el código que has decidido deliberadamente no rediseñar, reubicar y encapsular es justo lo correcto. La ley del doble salto sigue en pie: la reubicación es un movimiento limpio y aislado, y no finges que haya modernizado nada. La diferencia está en la honestidad sobre lo que compras.

La reescritura se reserva para el código que la merece: el núcleo del negocio, donde el cambio es constante, la rotación alta, la complejidad real, y donde tus desarrolladores sénior ya pasan la mayor parte del tiempo. Ahí, y solo ahí, el rediseño paga: rearquitecturar el núcleo compra agilidad, recorta deuda técnica y devuelve la mantenibilidad. Y ese núcleo reescrito debe diseñarse desde el primer día para convivir con el legado restante, con los patrones estratégicos de DDD y los de desplazamiento de legado, porque durante años tu parque será una federación de viejo y nuevo, no un reemplazo.

De ahí la inversión que recorre todo el libro vista desde la cartera. De casi todo lo que recuperas, decidirás racionalmente no reescribir nada. Recuperar no es el prelude obligado de reescribir; es lo que te permite no hacerlo donde no toca, y dirigir cada euro de rediseño al único sitio donde rinde. Para ese núcleo que sí decides reescribir, ahora sí toca decidir la arquitectura. Y ahí, por fin, el arquitecto elige. Su primera elección no es de código, es de mapa, y es el capítulo siguiente. Desde aquí, el resto del libro habla sobre todo de ese núcleo; lo encapsulado no sale de la historia, participa en el mapa por su contrato y reaparece cada vez que el nuevo tiene que convivir con él.

Capítulo 12

El mapa de contextos objetivo

El primer acto del rediseño no ocurre en el código. Ocurre en el mapa. Antes de escribir una línea del sistema nuevo, el arquitecto decide el mapa de contextos objetivo: qué contextos delimitados tendrá el parque resultante, qué modelo y qué lenguaje viven dentro de cada uno, y cómo se relacionan entre sí. **La primera elección de arquitectura no es de código, es de mapa.** Es el acto que la traducción se salta por completo, y es donde la modernización se gana o se pierde como diseño.

Recuerda que llegas aquí con dos mapas en la mano. El modelo recuperado trae el mapa as-is, la estructura tal como el sistema vivo la muestra, anotada como dato re-decidible. El mapa objetivo es otra cosa: es una decisión. Sobre un mismo dominio recuperado caben muchos mapas posibles, y elegir uno es juicio del arquitecto, no hallazgo del agente. Cada frontera que el mapa objetivo conserva del as-is es una decisión de conservar. Cada frontera que corrige es una decisión de corregir. Las dos se marcan como decisiones, con la disciplina de procedencia de siempre, porque el mapa objetivo no se recupera de ninguna parte: se elige.

Conviene precisar qué se decide al trazar un contexto, porque es más que un recuadro con nombre. Un contexto delimitado es la frontera dentro de la cual un modelo del dominio y su lenguaje son coherentes. Y el lenguaje ubicuo, conviene decirlo sin ambigüedad, no es uno para todo el negocio: es uno por contexto. Un mismo término puede significar cosas distintas en dos contextos, y esa diferencia no es un defecto a unificar, es información a proteger con una frontera. Forzar un lenguaje único sobre significados distintos produce el peor de los modelos, el que parece coherente y no lo es. El mapa existe para que cada significado tenga su casa.

CardDemo • paso 6/11

En CardDemo, «cuenta» no significa lo mismo para el servicing que la gobierna que para los informes que la agregan. Para el servicing es la línea de crédito viva, que protege su invariante; para los informes es una fila que se suma. El mismo nombre, dos modelos:

```
// Contexto Servicing: la cuenta protege su regla del ciclo
public sealed class Cuenta
{
    public Money CreditoDisponibleDelCiclo() =>
        limite - (cicloCredito - cicloDebito);
}

// Contexto Informes: la cuenta es una fila que se agrega
public sealed record FilaCuenta(long CuentaId, decimal Saldo);
```

El mapa objetivo les da contextos distintos en vez de un «cuenta» único que mienta a los dos.

El mapa incluye todos los contextos, no solo los que vas a reescribir. Aquí el triaje del capítulo anterior se vuelve cartografía. El COBOL encapsulado no es un residuo fuera del plano: es un contexto delimitado de pleno derecho, con

el modelo que tiene, su contrato y sus relaciones dibujadas. Un mapa que solo retrata lo nuevo es propaganda. El mapa honesto retrata la federación entera de viejo y nuevo que vas a operar durante años, porque las relaciones que más cuidado piden son justo las que cruzan esa costura.

Las relaciones entre contextos tienen nombres, y usarlos disciplina el diseño. El contexto que ofrece capacidades a muchos consumidores publica un servicio de host abierto con un lenguaje publicado. El contexto que consume y quiere proteger su modelo levanta una capa anticorrupción. No son alternativas entre las que elegir: son los dos extremos de la misma relación, el host abierto y el lenguaje publicado viven en el lado del que ofrece, la capa anticorrupción en el lado del que consume, y una relación sana puede tener los dos. Entre contextos cuyos equipos negocian prioridades hay una relación de cliente y proveedor. Donde aceptas el modelo del otro tal cual, sin traducir, eres conformista, que es barato y honesto cuando el modelo ajeno es bueno. Y donde integrar cuesta más de lo que vale, la relación correcta es caminos separados: duplicar un poco y no integrar también es una relación, la más barata de todas cuando aplica.

Un ejemplo pequeño deja ver cómo se combinan estas relaciones sobre un solo dominio.

CardDemo · paso 7/11

CardDemo deja ver el mapa objetivo en pequeño. El núcleo, servicing de cuentas y posting de movimientos, se reescribe, y el hotspot del fichero de cuentas se resuelve en el mapa antes que en el código: el estado de la cuenta pasa a tener un solo contexto dueño de su escritura, y los demás lo consumen por contrato. El interés, estable, queda encapsulado detrás de un host abierto con su lenguaje publicado. La identidad y el acceso, genéricos, se recortan pronto y se sustituyen por un servicio hospedado, consumido tras una capa anticorrupción. Los informes consumen aguas abajo con su propia capa anticorrupción. Tres movimientos de triaje y tres tipos de relación, un solo dominio, y ninguna de esas flechas estaba dictada.

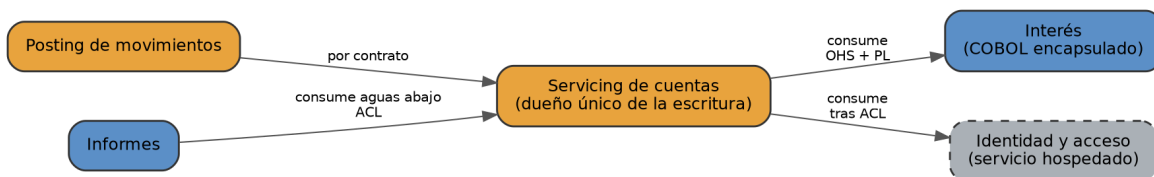


Figura. El mapa de contextos objetivo de CardDemo: el hotspot resuelto en un único contexto dueño de la escritura de la cuenta, con el resto consumiendo por contrato (OHS+PL del lado proveedor, ACL del lado consumidor).

El mapa tiene además un precio que el agente puede poner delante de cada opción. Cada frontera que redibuja respecto al as-is diverge del oráculo y encarece la verificación posterior; cada frontera que conserva la abarata. Un mapa ambicioso paga su ambición en la Parte IV. No es una razón para no ser ambicioso, es una razón para serlo con el precio a la vista, y para recordar que el mapa puede evolucionar después, con la red puesta, en vez de nacer perfecto.

Por eso el orden prudente fija el mapa primero y congela los contratos cuanto antes. Los contratos entre contextos son lo más caro de cambiar después, porque de ellos cuelgan los dos lados de cada costura. Mapa fijado, contratos publicados, y solo entonces el ascensor baja al interior de cada contexto que se reescribe. Ese interior es el diseño táctico, y es el capítulo siguiente.

Capítulo 13

El diseño táctico del núcleo

Con el mapa fijado y los contratos publicados, el ascensor baja una planta y entra en cada contexto del núcleo. Tienes un modelo del dominio, fiel, marcado, con la semántica separada de la estructura vieja, y un mapa que dice qué contextos se reescriben. Ahora decides cómo va a ser cada uno por dentro. Esto es diseño, en el sentido pleno, y por tanto es territorio del arquitecto, no del agente. El agente prepara la mesa de opciones, como vimos. El arquitecto elige el plato.

La primera idea liberadora es la que ya sembramos en el capítulo cuatro. **Sobre un mismo dominio recuperado caben muchas arquitecturas.** El dominio es uno. Las maneras de estructurarlo son muchas. El mapa ya agrupó los contextos; dentro de cada uno puedes elegir un estilo de arquitectura u otro, el paradigma, procedural, orientado a objetos o funcional, y decidir qué partes merecen riqueza de modelo y cuáles pueden quedarse simples. Ninguna de esas decisiones está dictada por el dominio. Todas son juicios del arquitecto sobre qué sirve mejor a las restricciones de hoy, mantenibilidad, rendimiento, autonomía de equipos, lo que importe en tu caso. Esta multiplicidad es exactamente el premio de haber separado la semántica de la estructura. Si hubieras conservado la estructura vieja, no habría elección. Como la has soltado, hay un abanico.

El trabajo de diseño se divide en dos mitades de naturaleza muy distinta, y separarlas evita mucha confusión. Hay una mitad mecánica y una mitad de elección.

La mitad mecánica es la traducción de patrones del dominio recuperado a patrones de la arquitectura objetivo, cuando esa traducción es directa. Una operación del dominio se convierte en una unidad de ejecución de la arquitectura nueva. Un evento del dominio se convierte en un mensaje. Un proceso por lotes se convierte en un flujo. Estas correspondencias son bastante mecánicas una vez decidido el estilo de arquitectura, y son justo el tipo de trabajo que el agente hace bien y rápido. Aquí los frameworks concretos entran como ejemplos. Un framework que organiza el código en cortes verticales, cada uno alrededor de una operación de negocio y no de una capa técnica, es un ejemplo de arquitectura objetivo que encaja de forma natural con un dominio recuperado en términos de operaciones y reglas, porque el corte vertical preserva la cohesión de cada operación. Pero el framework es el ejemplo. El concepto durable es que hay una parte de la traducción a la arquitectura nueva que es mecánica y delegable.

En la práctica. En el ecosistema .NET, un framework de arquitectura por cortes verticales como Wolverine organiza cada operación de negocio en su propio corte, con su mensaje y su manejador, en lugar de repartirla entre capas técnicas. Una operación del dominio se traduce en un corte con su mensaje de entrada y su manejador; un evento del dominio, en un mensaje; un cálculo por lotes, en un flujo. Otros estilos, como el hexagonal, llegan al mismo sitio por otro camino. El framework es la instancia; lo durable es que el corte sigue la operación de negocio, no la capa técnica.

CardDemo • paso 8/11

La operación de CardDemo que autoriza un movimiento está hoy repartida entre el posting y el REWRITE del fichero. En el diseño nuevo se vuelve un solo corte vertical, un mensaje y su manejador:

```
public record AutorizarMovimiento(long CuentaId, Money Importe);

public async Task<Resultado> Handle(AutorizarMovimiento c)
{
    var cuenta = await cuentas.Cargar(c.CuentaId);
    return cuenta.Autorizar(c.Importe); // aplica las reglas del ciclo
}
```

El cálculo de interés por lotes, que recorre las cuentas al cerrar el ciclo, se vuelve un flujo aparte. Cada corte sigue una operación de negocio, no una capa técnica.

La mitad de elección es donde vive el arquitecto. Aquí están las decisiones que el dominio no dicta. ¿Conviertes un concepto que el sistema viejo tenía anémico, un saco de datos sin comportamiento, en un agregado rico que protege sus propios invariantes? ¿Redibujas una frontera que el sistema viejo tenía mal puesta? ¿Conviertes un dato suelto en un objeto de valor con significado propio, como el saldo de la cuenta, hoy un campo numérico en el fichero, en un importe que lleva siempre su moneda y se niega a sumarse con otra distinta? Cada una de estas es una elección de diseño, y cada una tiene consecuencias, no solo sobre la calidad del sistema nuevo sino, crucialmente, sobre lo cara que será la verificación posterior. Una arquitectura más fiel a la estructura vieja es más barata de verificar, porque se parece más al oráculo. Una arquitectura más ambiciosa, que redibuja fronteras, es más cara de verificar, porque diverge más. El agente puede y debe cuantificar esa consecuencia para cada opción. El arquitecto decide con esa información sobre la mesa.

Hay una decisión recurrente que ilustra bien por qué el juicio humano es insustituible aquí, y es la reubicación de un invariante. En el sistema viejo, una regla podía estar aplicada en varios sitios a la vez, repetida, porque varias partes del sistema escribían el mismo dato y cada una comprobaba la regla por su cuenta. Al rediseñar, lo natural es reubicar esa regla en un solo lugar, el agregado que es dueño del dato, para que la proteja una vez y bien. Mecánico, en apariencia. Pero hay una trampa que solo el juicio detecta. Reubicar la regla en un sitio único asume que todas las copias viejas de la regla eran idénticas. ¿Y si no lo eran? ¿Y si una de las partes que escribía el dato aplicaba la regla y otra no, deliberadamente, porque en ese flujo concreto la regla no debía aplicarse? Entonces consolidar las copias en una sola no es una limpieza, es un cambio de comportamiento, y posiblemente un error grave. Detectar esa divergencia exige mirar el dominio recuperado con ojo crítico, y decidir si las copias eran la misma regla mal repetida o reglas distintas que parecían iguales. Eso es juicio. El agente puede señalarte que hay varias copias y mostrarte si difieren. Decidir qué significa esa diferencia y qué hacer con ella es del arquitecto, y es exactamente la clase de decisión que, si la delegas, te explota meses después.

Hay un orden prudente para todo esto, y conecta de nuevo con el 3X. No persigas el modelo ambicioso en el primer intento. La capa estratégica ya quedó resuelta en el capítulo anterior: el mapa fijado, el sistema viejo envuelto en su capa anticorrupción, los contratos publicados. Por dentro, empieza conservador. Cada contexto se queda fino, casi un calco del comportamiento que muestra el oráculo, aunque sea anémico y poco elegante. Ese diseño es barato de verificar, porque se parece al oráculo, así que cierras la equivalencia rápido y tienes un sistema nuevo que demostradamente hace lo mismo que el viejo. Solo entonces, con la red de seguridad puesta, abor das las mejoras ambiciosas, enriquecer el modelo táctico de cada contexto y, si hace falta, volver a subir al mapa a redibujar una frontera, cada paso con su propia verificación.

La especificación del dominio, que veremos en la parte siguiente, sirve de gate a los dos diseños sin cambiar una coma. Da igual que verifiques el conservador o el ambicioso: como está escrita en términos del dominio y no de ninguna arquitectura, mide a los dos con la misma vara. Por eso puedes empezar conservador, cerrar la equivalencia, y enriquecer después sin reescribir la especificación, porque lo que mide es el qué, que no cambia, no el cómo, que sí. Primero demuestra que no rompes nada con un diseño humilde. Después mejora, paso a paso, sin soltar nunca la red. Optimizar antes de tener la red es, otra vez, explorar donde tocaba estabilizar.

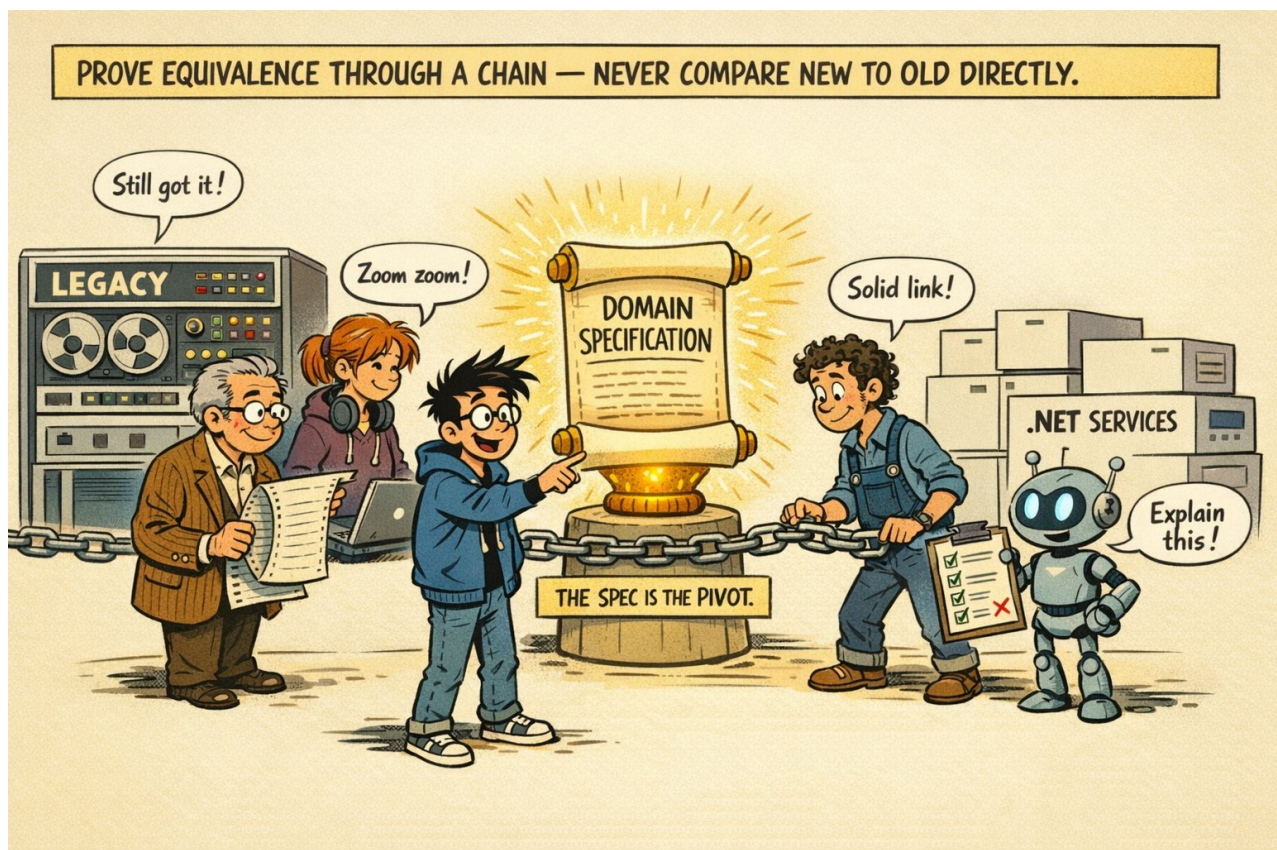
Esto nos lleva directos al pilar que hace que todo lo anterior sea seguro en vez de temerario. Has recuperado un dominio y has decidido una arquitectura nueva que diverge de la vieja. ¿Cómo demuestras, no asumes, demuestras, que el sistema nuevo significa lo mismo que el viejo? Esa es la Parte IV, y es donde el método deja de ser una buena intención y se vuelve un compromiso verificable.

Parte IV

Probar que no lo rompiste

Capítulo 14

La cadena transitiva



Aquí está el problema en su forma más cruda. Tienes el sistema viejo, que funciona pero cuyo diseño aborreces. Quieres un sistema nuevo, con un diseño que has elegido y que diverge a propósito del viejo. Y tienes que demostrar que el nuevo se comporta igual que el viejo en todo lo que importa. La tentación es obvia. Compara el nuevo con el viejo, directamente. Misma entrada a los dos, compara las salidas, si coinciden, equivalentes.

Esa comparación directa es más difícil de lo que parece, y a veces imposible de hacer bien, por una razón que viene del capítulo cuatro. El sistema nuevo y el viejo divergen por dentro a propósito. Tienen fronteras distintas, estructuras de datos distintas, puede que hasta hablen idiomas distintos en sus interfaces, porque el viejo habla en términos de su mecanismo antiguo y el nuevo en términos de su dominio limpio. Comparar dos sistemas que ni siquiera presentan la misma forma de entrada y salida exige una capa de traducción entre ellos, y esa capa de traducción es código nuevo, no verificado, que se interpone justo en el sitio donde quieres certeza. Acabas confiando tu prueba de equivalencia a un traductor cuya corrección nadie ha demostrado. La comparación directa mete una incógnita en el centro de la prueba.

CardDemo • paso 9/11

El CardDemo viejo habla por una COMMAREA posicional, el copybook C0COM01Y, donde cada dato vive en su sitio por desplazamiento:

01 CARDDEMO-COMMAREA.

05 CDEM0-ACCT-ID PIC 9(11).

05 CDEM0-ACCT-STATUS PIC X(01).

El nuevo habla por un objeto de dominio:

```
public record MovimientoAutorizado(long CuentaId, Money CreditoRestante);
```

Dicen lo mismo en dos idiomas que no se parecen. Compararlos en directo exigiría un traductor entre los dos, código sin verificar puesto justo donde quieres certeza. Por eso la cadena no los compara entre sí: los conecta a través de la especificación, escrita en lenguaje de dominio, que sirve de patrón a los dos.

La solución es un rodeo que resulta ser más recto que la línea directa. **No compares nunca el sistema nuevo contra el viejo en directo. Construye una cadena de equivalencias transitiva.** La idea es introducir pasos intermedios elegidos de forma que cada eslabón de la cadena sea fácil de verificar, y dejar que la transitividad haga el resto. Si A es equivalente a B, y B es equivalente a C, entonces A es equivalente a C, sin haber comparado nunca A con C directamente.

Veamos la cadena. El primer eslabón es construir una réplica del sistema viejo que preserve su estructura, no su diseño, su estructura. Esto es exactamente el oráculo del capítulo tres, la copia que corre rápido y local. Como preserva la estructura del viejo, demostrar que esa réplica es equivalente al sistema viejo original es fácil, porque son casi iguales por dentro, así que la comparación es casi una identidad y se puede hacer con una verificación fina, comparando incluso detalles internos, porque las estructuras se corresponden. Este eslabón es barato precisamente porque no hay divergencia de diseño que salvar. Una vez cerrado, tienes una réplica fiel del comportamiento viejo que vive en tu máquina y responde en milisegundos. Y, dato importante, a partir de aquí el sistema viejo original puede retirarse de la ecuación del trabajo diario, porque su comportamiento ya vive, fielmente, en la réplica.

El segundo eslabón no compara sistemas, captura comportamiento. Usas la réplica fiel como oráculo y observas, sistemáticamente, qué hace ante un conjunto bien elegido de entradas. El resultado de esa observación lo escribes como una especificación ejecutable, una colección de casos que dicen, ante esta situación y esta petición, el comportamiento observable es este. Esa especificación es el siguiente eslabón de la cadena, y tiene una propiedad mágica. Está expresada en términos del dominio, no en términos de la estructura de ningún sistema. No dice lo que hace el sistema viejo por dentro ni lo que hará el nuevo por dentro. Dice qué se observa, en lenguaje de dominio. Por eso puede servir de patrón para los dos.

El tercer eslabón es el sistema nuevo. Lo construyes, con su diseño divergente, y demuestras que satisface la especificación capturada. No lo comparas con el viejo. Lo comparas con la especificación, que está en lenguaje de dominio y por tanto no exige una capa de traducción dudosa, solo exige que el sistema nuevo, expresado en su propio dominio limpio, produzca los observables que la especificación dicta.

Y ahí se cierra la transitividad. El sistema viejo es equivalente a su réplica fiel, eslabón uno, verificación fina. La réplica fiel produce la especificación, eslabón dos, por observación directa. El sistema nuevo satisface la especificación, eslabón tres. Por tanto el sistema nuevo es equivalente al viejo, sin haberlos comparado jamás de forma directa, sin haber construido jamás esa capa de traducción dudosa entre dos sistemas que divergen por dentro. La especificación, en lenguaje de dominio, es el pivote que conecta los dos mundos sin tocarlos entre sí.

Hay una elegancia adicional en esta cadena que conviene notar. La granularidad de la verificación sigue al parecido estructural de cada eslabón, y esto es un principio general que merece recordarse. Donde dos cosas se parecen mucho por dentro, eslabón uno, viejo contra réplica, puedes y debes verificar fino, mirando detalles internos, porque la correspondencia existe. Donde dos cosas divergen por dentro a propósito, eslabón tres, especificación contra sistema nuevo, no puedes mirar detalles internos porque no se corresponden, así que verificas solo lo observable desde fuera, y lo observable se nombra en el lenguaje del dominio, en el espacio del problema, no en términos técnicos de ninguna solución. La granularidad de la prueba se ajusta a cuánto se parecen las cosas que comparas. Forzar una comparación fina entre cosas que divergen es el error que la cadena transitiva evita.

La cadena reconcilia además el triaje con la verificación, y la imagen es limpia: **el triaje decide cuánta cadena recorres.**

Para el COBOL que decidiste encapsular y reubicar, el primer eslabón es toda la verificación que necesita: demostrada la fidelidad de la réplica recompilada, ese contexto ya está donde tenía que estar, detrás de su contrato. Solo el núcleo que reescribes recorre la cadena entera, porque solo él diverge por diseño. La cadena no es un peaje uniforme; es un camino del que cada carga recorre el tramo que su destino exige.

Queda por desarrollar qué es exactamente esa especificación capturada, cómo se escribe para que sea a la vez patrón de prueba, red de regresión y documentación, y por qué su independencia del idioma de los dos sistemas es lo que protege tu libertad de rediseño. Ese es el siguiente capítulo.

Capítulo 15

El comportamiento como especificación ejecutable

La especificación capturada es el pivote de toda la verificación, así que merece un capítulo entero sobre qué es y por qué tiene la forma que tiene. La idea central es que el comportamiento observado del oráculo se escribe como una especificación ejecutable, una colección de casos que una máquina puede ejecutar y comprobar, no un documento en prosa que alguien tiene que interpretar.

La forma más útil que conozco para esto es la del ejemplo tabulado, en el estilo de dado esto, cuando aquello, entonces esto otro. Cada caso describe un estado de partida, dado esto, una acción, cuando aquello, y el observable esperado, entonces esto otro. Una fila de la tabla es un caso. La tabla entera es la especificación de una operación del dominio. Esta forma tiene una virtud pedagógica enorme, porque se lee casi como lenguaje natural y la puede revisar un experto del negocio que no programa, y a la vez es ejecutable, porque cada fila se puede correr contra un sistema y comprobar si el observable real coincide con el esperado. Esta idea no es nueva. Ward Cunningham la introdujo en 2002 con FIT, Framework for Integrated Test: tablas que un experto del negocio puede leer y que a la vez una máquina ejecuta.

Lo más importante de la especificación, y lo que la hace el pivote de la cadena transitiva, es el idioma en que está escrita. **La especificación se escribe en el lenguaje ubicuo del dominio, no en el idioma de ninguno de los dos sistemas.** El sistema viejo habla en términos de su mecanismo, sus estructuras de memoria, sus códigos técnicos. El sistema nuevo habla en términos de su dominio limpio. Si escribieras la especificación en el idioma del viejo, atarías el sistema nuevo a la forma vieja y perderías la libertad de rediseño que tanto te costó ganar. Si la escribieras en el idioma del nuevo, no podrías usarla para verificar contra el viejo. La única forma que sirve para los dos es un tercer idioma, el del dominio puro, independiente de ambas implementaciones. Ese es el idioma en que el experto del negocio describiría la regla sin saber nada de ordenadores. Con una precisión que el capítulo doce dejó establecida: el lenguaje ubicuo es uno por contexto, no uno para todo el negocio. Cada contexto se especifica en su propio lenguaje, y los casos que cruzan contextos hablan el lenguaje publicado del contrato que los une.

Esto tiene una consecuencia arquitectónica concreta. Como la especificación está en lenguaje de dominio y cada sistema habla su propio idioma, hace falta una pieza de traducción por sistema, que tome un caso en lenguaje de dominio y lo ejecute contra ese sistema concreto, traduciendo la entrada al idioma del sistema y la salida de vuelta al dominio. Hay una de estas piezas para el viejo y otra para el nuevo. La virtud de aislar la traducción en estas piezas es que la dependencia del idioma de cada sistema queda confinada ahí, en un sitio pequeño y bien delimitado, tan acotado que se puede probar por unidad y a fondo, agotando sus casos, en vez de contaminar la especificación. La especificación permanece pura, en dominio, reutilizable, y la suciedad de cada idioma concreto vive aislada en su pieza de traducción. Esa pureza es lo que protege tu libertad de rediseño, porque puedes cambiar entero el sistema nuevo, su idioma incluido, su stack tecnológico entero, y la especificación sigue valiendo, solo cambias su pieza de traducción.

La especificación captura el observable completo del dominio, y hay que ser cuidadoso con qué significa completo. Significa no solo la respuesta directa a la petición, sino también los efectos colaterales que importan al dominio, el cambio de estado que la operación deja tras de sí. Si una operación además de responder modifica el estado de una entidad, ese cambio de estado es parte del observable y debe estar en la especificación. Pero, y esto es clave, esos efectos

colaterales se observan a través de consultas de dominio, preguntándole al sistema por su estado en términos de dominio, no espiando su almacén de datos por dentro. Espiar el almacén rompería la caja negra y volvería a atar la prueba a la estructura concreta de almacenamiento, prohibiendo el rediseño. Se pregunta por el estado en lenguaje de dominio, igual que se preguntó por la respuesta.

CardDemo • paso 10/11

En CardDemo, autorizar un movimiento descuenta el crédito disponible. Para comprobar el efecto no se lee el fichero de cuentas: se le pregunta al sistema por el saldo en lenguaje de dominio y se comprueba que bajó lo que debía. Leer el almacén ataría la prueba a la estructura de almacenamiento; preguntar por el dominio la deja libre.

```
// rompe la caja negra: ata la prueba al almacenamiento
var saldo = LeerFicheroCuentas(idCuenta).CampoSaldo;

// observable de dominio: pregunta en lenguaje de dominio
var saldo = await cuentas.ObtenerCreditoDisponible(idCuenta);
```

Hay un límite honesto que conviene declarar. Completo significa todo el observable de dominio que importa, no literalmente todo efecto. Los efectos puramente técnicos, una entrada en un registro de log, un contador interno, un código de estado de la fontanería, no se capturan, porque no son dominio, son mecanismo, y compararlos sería comparar precisamente lo que tienes derecho a cambiar. Y hay un límite más profundo, solo puedes capturar lo que el sistema expone. Si hay un efecto de dominio que ninguna consulta saca a la luz, queda fuera de la especificación, es un punto ciego. Capturar todo lo observable es el objetivo, pero exige que el sistema exponga, vía consultas de dominio, todos sus efectos de dominio, y eso no viene gratis, hay que diseñarlo.

Para terminar, una observación sobre por qué esta especificación es tan rentable. Cada fila de la tabla vive tres vidas a la vez. Mientras el sistema viejo o su réplica siguen vivos, la fila es un caso de prueba de paridad, comprueba que viejo y nuevo coinciden. Cuando el viejo se jubila, la misma fila se convierte en una prueba de regresión del sistema nuevo, comprueba que ningún cambio futuro rompe el comportamiento heredado. Y siempre, desde el primer día, la fila es documentación viva del dominio, en lenguaje que el negocio entiende, y que los agentes también entienden y pueden volcar de muchas formas, en tablas, en markdown, en diagramas. No miente, porque es ejecutable y se rompería si mintiera. Pocas cosas en ingeniería dan tres usos por el precio de uno. La especificación de dominio es una de ellas, y por eso vale la pena el esfuerzo de capturarla bien.

Una última pieza falta para que la verificación sea sólida, y es la más sutil de todas. Qué comparas exactamente cuando comparas el observable, y en qué orden haces las cosas. Eso es el último capítulo de esta parte.

Capítulo 16

Paridad caja negra y la ley de orden

Ya sabemos que comparamos lo observable desde fuera, no las tripas. A esa forma de verificar la llamamos paridad caja negra, porque trata cada sistema como una caja cerrada de la que solo se ven las entradas y las salidas. Pero caja negra no significa comparar a ciegas todo lo que sale. Significa elegir con cuidado qué de lo que sale cuenta como observable de dominio y qué es ruido técnico que hay que ignorar. Esa elección, qué se compara, es el dial más delicado de toda la verificación, y calibrarlo mal arruina la prueba en una de dos direcciones opuestas.

Si comparas demasiado, incluyendo en la comparación detalles técnicos que el sistema nuevo tiene derecho a hacer distinto, la prueba falla constantemente por diferencias que no importan, un código interno distinto, un formato de fecha distinto, un orden de campos distinto. Te ahogas en falsos positivos, la prueba se vuelve ruido, y acabas ignorándola, que es lo peor que le puede pasar a una prueba. Si comparas demasiado poco, excluyendo cosas que sí eran observables de dominio, la prueba pasa aunque el comportamiento haya cambiado de verdad, y te da una falsa seguridad, que es todavía peor, porque te deja romper el sistema con la conciencia tranquila. Entre comparar de más y comparar de menos hay una calibración correcta, y encontrarla es el oficio.

La regla de calibración es esta. Clasifica cada campo del observable en una de varias categorías. Los resultados de dominio y el estado de dominio resultante son comparación estricta, tienen que coincidir exactamente, hasta el último céntimo, porque son la semántica que prometiste preservar. Los campos puramente técnicos, mecanismo sin consecuencia de dominio, se excluyen, porque son justo lo que tienes derecho a cambiar. Los campos no deterministas, un identificador generado, una marca de tiempo, se normalizan antes de comparar, se reducen a una forma canónica que ignora lo que legítimamente varía. Y hay una categoría peligrosa, los campos ambiguos o entrelazados, donde un valor técnico esconde una consecuencia de dominio, y esos hay que desenredarlos con cuidado, comparando la consecuencia de dominio y no el valor técnico que la transporta. Y cuidado con lo que parece técnico y no lo es: algunos campos parecen mecanismo, pero si alguien aguas abajo depende de ellos son reglas de dominio, y van a comparación estricta.

CardDemo • paso 11/11

Dos campos de CardDemo parecen mecanismo y no lo son. El orden de los campos del extracto que Informes consume, y el formato del grupo de pricing que gobierna la búsqueda de la tasa, los lee alguien aguas abajo, así que son reglas de dominio y van a comparación estricta. El saldo de la cuenta tras cerrar el ciclo se compara hasta el último céntimo. Y el caso límite se escribe como una fila de especificación en lenguaje de dominio:

Dado un crédito disponible de 100,00 en el ciclo en curso

Cuando se autoriza un movimiento de 100,01

Entonces el movimiento se rechaza por exceder el crédito del ciclo

La regla de oro de esta clasificación es comparar estricto por defecto y excluir solo con evidencia. La carga de la prueba recae sobre la exclusión, no sobre la inclusión. Si dudas de si un campo importa, lo comparas, porque el riesgo de comparar de más es ruido molesto, mientras que el riesgo de excluir de más es ceguera peligrosa. Solo excluyes un campo cuando puedes demostrar que es técnico y que no arrastra ninguna consecuencia de dominio. Y esa demostración debería poder rastrearse, cada campo excluido traza a una pieza de mecanismo concreta sin efecto de dominio. Excluir por comodidad,

porque el campo molesta, es abrir la puerta a la ceguera.

El caso de los errores ilustra bien la sutileza. Cuando una operación falla, ¿comparas el modo de fallo o el desenlace de dominio? La respuesta correcta casi siempre es el desenlace de dominio. Si el sistema viejo rechaza una operación por una regla de negocio, y el nuevo la rechaza por la misma regla de negocio, son equivalentes, aunque por dentro el mecanismo del rechazo sea completamente distinto. Lo que importa es que la operación se rechazó por esa razón de dominio, no cómo se implementó el rechazo. En cambio, si el viejo falla por una avería técnica, eso no es dominio, es mecanismo, y no debería formar parte de la comparación. Distinguir el rechazo de negocio del fallo técnico es exactamente el desenredo del capítulo cuatro, donde advertíamos que ahí se esconde la pérdida silenciosa de semántica. La paridad bien calibrada es donde esa distinción se vuelve operativa.

Falta el segundo tema del capítulo, y es una regla de orden que tiene rango de ley en este método. **La equivalencia se cierra antes de que empiece cualquier refactor de calidad.** Es decir, primero demuestras que el sistema nuevo es equivalente al viejo, con su diseño conservador inicial, y solo después, con esa equivalencia probada y en verde, empiezas a mejorar la calidad interna del sistema nuevo, a refactorizar, a enriquecer, a pulir. No al revés. Nunca al revés.

La razón es estructural, no de preferencia. Refactorizar es cambiar la estructura preservando el comportamiento. Para hacerlo con seguridad necesitas una red que te diga si has roto el comportamiento mientras cambiabas la estructura. Esa red es justo la especificación de equivalencia. Si refactorizas antes de tener la equivalencia cerrada, refactorizas sin red, y un cambio que rompe el comportamiento pasa inadvertido, porque no tienes nada que lo detecte. Peor aún, si haces las dos cosas a la vez, rediseñar para equivalencia y refactorizar para calidad mezcladas, cuando algo falle no sabrás si falló porque tu rediseño no era equivalente o porque tu refactor rompió algo. Habrás perdido la capacidad de diagnosticar, que es el mismo pecado del doble salto del capítulo dos, hacer dos cambios distintos en el mismo movimiento.

Esta ley de orden tiene una autoridad que conviene encarnar en algo, un guardián que la haga cumplir. Llámalo como quieras, lo importante es su función. Es la autoridad de equivalencia. Posee la especificación, que es su evidencia. Corre las comparaciones. Custodia las corridas en verde, con su cobertura y su procedencia, para que conste qué se ha probado y con qué fuerza. Y hace cumplir el orden, no deja empezar el refactor de calidad hasta que la equivalencia está cerrada. No es solo una herramienta que genera pruebas. Es el árbitro que garantiza que el método se sigue en el orden correcto, porque la historia de los desastres de modernización está llena de equipos que refactorizaron sobre arena, sobre código que creían equivalente sin haberlo probado, y que descubrieron en producción que no lo era.

Con la verificación entera en pie, cadena transitiva, especificación de dominio como pivote, paridad caja negra bien calibrada, y la ley de orden custodiada, tienes la red de seguridad que hace que el rediseño sea ingeniería y no apuesta. Pero todavía falta la parte más larga y menos glamurosa de cualquier modernización real. El sistema viejo no se apaga el día que el nuevo está listo. Los dos conviven, a veces durante años. Convivir bien es un arte propio, y es la Parte V.

Parte V

Convivir con dos sistemas

Capítulo 17

El estrangulador, honesto

Casi todo lo escrito sobre modernización trata el cambio del viejo al nuevo como un instante, el día del cambio, como si hubiera un interruptor. En la realidad de cualquier sistema que importe, no hay interruptor. Hay un periodo, a menudo largo, en el que el sistema viejo y el nuevo conviven, atendiendo entre los dos la operación del negocio, mientras el nuevo va asumiendo responsabilidades poco a poco. Diseñar esa convivencia es trabajo de arquitectura de primera clase, no un detalle de despliegue, y subestimarlos es una de las causas más comunes de que una modernización técnicamente correcta acabe en desastre operativo. Y la convivencia no es solo entre el viejo y el núcleo nuevo: el parque que sale del triaje incluye contextos encapsulados que no migran, conviven para quedarse, detrás de su contrato. El estrangulador gobierna la parte que sí se desplaza, el núcleo que se reescribe.

El patrón clásico para esto es el estrangulador, que Martin Fowler nombró por una higuera que crece alrededor de un árbol hasta sustituirlo. La idea es poner una fachada delante de todo, que reciba todas las peticiones, y que para cada una decida si la atiende el sistema viejo o el nuevo. Al principio casi todo va al viejo. Poco a poco, a medida que cada parte del nuevo se prueba equivalente, la fachada empieza a enrutar esa parte al nuevo. La higuera va creciendo. El árbol viejo va menguando. Un día el viejo ya no atiende nada y se puede retirar.

Descrito así suena limpio, y la parte de enrutar de verdad lo es. La fachada que decide a quién mandar cada petición es, conceptualmente, sencilla. El problema, y aquí viene la honestidad del título, es que enrutar es la parte fácil y visible, mientras que la parte difícil e invisible está debajo, en los datos. Y casi toda la literatura se entusiasma con la parte fácil y calla la difícil.

Veamos por qué los datos son el problema. Mientras los dos sistemas conviven, hay un dato del negocio, pongamos el estado de una entidad central, que algunas operaciones ya atiende el sistema nuevo y otras todavía el viejo. Si una operación que ya migró escribe ese dato en el almacén nuevo, y otra que no ha migrado lo escribe en el almacén viejo, el dato se ha partido en dos. Hay dos versiones, en dos almacenes, divergiendo. El sistema nuevo cree una cosa sobre la entidad, el viejo cree otra, y el negocio está corrupto sin que nadie lo note hasta que es tarde. La fachada que enruta operaciones no resuelve esto en absoluto, porque el problema no está en las operaciones, está en el estado que comparten.

De aquí sale el principio que gobierna el troceado, y es menos obvio de lo que parece. **Corta a lo largo de la propiedad de la escritura, no donde te resulte cómodo.** Es decir, agrupa para migrar juntas todas las operaciones que escriben un mismo dato. Si cuatro operaciones distintas escriben el estado de la entidad central, esas cuatro migran a la vez, en el mismo paso, o no migran ninguna. Si migras dos y dejas dos en el viejo, partes el estado y lo corrompes. La frontera por la que cortas tiene que respetar quién es dueño de escribir cada dato, porque solo así garantizas que en cada momento un dato lo escribe un único sistema, y por tanto no puede divergir.

Esto explica un orden de troceado que de otro modo parecería arbitrario. Las partes del sistema que son periféricas, que leen mucho y escriben poco o escriben datos que solo ellas tocan, son fáciles de migrar primero, porque cortarlas no parte ningún estado compartido. Las partes centrales, donde varias operaciones se pelean por escribir el mismo estado crítico, son las que hay que migrar al final y en bloque, porque están enredadas por la propiedad de la escritura. El núcleo se migra entero o no se migra, y se deja para el final precisamente porque es el nudo. Quien intenta migrar el núcleo a trocitos, una operación hoy y otra el mes que viene, está partiendo el estado más crítico del negocio en cada paso. Es el camino más corto al desastre silencioso.

La fachada, además, hace doble servicio si la diseñas bien. La misma pieza de traducción que en la verificación convertía entre el idioma del dominio y el de cada sistema sirve aquí de puente entre los dos sistemas durante la convivencia. Es un ejemplo de arquitectura transitoria, en el sentido del catálogo de desplazamiento de legacy de Martin Fowler: andamiaje que se instala para sostener el reemplazo y que se retira en cuanto deja de hacer falta. No es casualidad. Es la economía del método premiando la coherencia. Una pieza bien diseñada para aislar la dependencia de un idioma sirve igual para verificar que para convivir. Volveremos a verla en el capítulo siguiente, donde la convivencia se vuelve la última prueba antes del cambio definitivo.

Por ahora, la lección es sobria. El estrangulador es un patrón excelente, pero su parte famosa, enrutar, es la fácil. La parte que decide si tu modernización vive o muere es la coexistencia de datos, y se gobierna con una regla: cortar por la propiedad de la escritura, para que ningún dato tenga dos dueños a la vez. Si interiorizas solo una cosa de este capítulo, que sea esa.

Capítulo 18

Cutover sin fe

Llega el momento de cambiar de verdad una parte del sistema, de dejar de enrutarla al viejo y enrutarla al nuevo de forma definitiva. A ese momento se le llama cutover, el corte. Y la pregunta es, cuándo es seguro hacerlo. La respuesta perezosa, cuando las pruebas pasan, es insuficiente y peligrosa, porque que las pruebas pasen una vez en tu entorno no te dice cómo se comportará el sistema nuevo ante el tráfico real, con toda su variedad y sus casos raros que tus pruebas quizá no cubren.

El método propone un criterio de cutover que no se apoya en la fe sino en la evidencia acumulada, y tiene varias condiciones que deben cumplirse a la vez. La primera, la especificación de equivalencia está en verde, el sistema nuevo coincide con el oráculo en todo el observable de dominio estricto. La segunda, la cobertura de esa especificación es suficiente, ha ejercitado las ramas, las fronteras, los casos límite, no solo el camino feliz. Una especificación en verde con cobertura pobre es una falsa tranquilidad, porque solo prueba que coinciden donde miraste, y donde no miraste viven los errores. La tercera condición es la decisiva y la que más se salta la gente con prisa.

La tercera condición es correr el sistema nuevo en sombra sobre tráfico real, durante un tiempo, y comprobar que coincide con el viejo. Correr en sombra significa que el tráfico de producción real va a los dos sistemas a la vez, pero solo cuenta la respuesta del viejo, que es el que sigue al mando. El nuevo procesa el mismo tráfico en paralelo, su respuesta se compara con la del viejo, pero no se usa, no afecta al negocio. Es un ensayo con red, el nuevo actuando sobre la realidad sin consecuencias. Durante ese periodo acumulas evidencia de que el nuevo coincide con el viejo no sobre los casos que a ti se te ocurrieron, sino sobre la distribución real de lo que de verdad pasa, incluidos los casos raros que ni tu especificación ni tu imaginación habían cubierto. La sombra es donde aflora la pérdida silenciosa de semántica que se escapó a la captura, porque el tráfico real es el examinador más exhaustivo que existe, lo prueba todo, sin piedad y sin avisar.

La cuarta condición cierra el criterio. Toda divergencia observada en sombra se ha investigado y resuelto, o se ha ratificado como aceptable. Una divergencia en sombra es oro, es el sistema avisándote de un caso donde nuevo y viejo no coinciden. Cada una merece investigación. Casi siempre revela un fallo del nuevo que arreglas. A veces revela que la divergencia es deseada, un cambio de regla que decidiste a propósito, y entonces se ratifica y se documenta como tal. Lo que nunca se hace es ignorarla. Cutover con divergencias de sombra sin explicar es cutover con fe, justo lo que el método rechaza.

Hay una lección histórica detrás de toda esta cautela, y el capítulo uno le puso nombres. Las modernizaciones que han acabado en titulares de desastre, sistemas que corrompieron datos masivamente tras el cambio, comparten casi siempre la misma raíz. No fue un fallo de herramienta. Fue un fallo de gobierno. Alguien afirmó la equivalencia en vez de demostrarla, cortó sobre esa afirmación, y la realidad cobró la diferencia. El criterio de cutover sin fe, gate verde más cobertura más sombra más divergencias resueltas, es la respuesta directa a esa clase de desastre. No cortas sobre lo que crees. Cortas sobre lo que has demostrado y confirmado contra la realidad.

Hay una quinta condición, y el cementerio del capítulo uno la justifica solo. El corte necesita una vuelta atrás ensayada: un camino de retorno al sistema viejo que se ha probado, no solo escrito, con sus datos y sus plazos claros. El desastre de la nómina australiana no fue solo cortar mal; fue cortar sin salida. **El corte es reversible hasta que deja de serlo, y esa frontera se cruza a sabiendas, no se descubre.** Mientras la vuelta atrás existe, una sorpresa es un incidente; cuando deja de existir, la misma sorpresa es un desastre. Saber en qué punto exacto del plan se quema esa nave, y decidir cruzarlo con evidencia y no con calendario, es parte del criterio.

Y una honestidad más, que conecta con la primera lección del cementerio. Este libro se concentra en la equivalencia de comportamiento, pero el corte también mueve estado: la carga inicial de los datos, su conciliación contra el origen, los recuentos que cuadran. Esa disciplina tiene su propio oficio y este método no lo detalla, pero sí lo exige en el gate: la conciliación de datos en verde es condición del corte tanto como la paridad en verde, porque la paridad de datos y la de comportamiento son dos pruebas distintas, y el cementerio está lleno de quien presentó una como si fuera la otra.

Cuando todas las condiciones se cumplen, el corte puede hacerse, y conviene hacerlo gradual incluso entonces. En vez de mandar de golpe todo el tráfico al nuevo, le mandas una fracción pequeña como sistema al mando, vigilas, y si aguanta, subes la fracción poco a poco. Es la prudencia llevada hasta el último metro. Incluso con toda la evidencia acumulada, un cambio gradual te deja una salida si algo inesperado aparece con el sistema nuevo de verdad al mando.

Una advertencia técnica honesta para no vender la sombra como gratis. Correr en sombra exige que el sistema nuevo procese tráfico real sin causar efectos reales en el mundo. Sus escrituras tienen que ir a un almacén de sombra, no al de producción. Sus efectos externos, mandar una notificación, llamar a un tercero, tienen que estar suprimidos, porque si no, el ensayo se vuelve realidad. Para operaciones de puro cálculo y cambio de estado interno, esto es limpio. Para operaciones con efectos externos visibles, es más delicado, no puedes de verdad mandar la notificación dos veces, así que suprimes el efecto externo y comparas solo la decisión interna que lo habría disparado. Diseñar bien esa supresión es parte del coste de la sombra, y conviene presupuestarlo en vez de descubrirlo tarde.

Con el cutover hecho, parte por parte, el sistema nuevo va asumiendo el mando y el viejo va quedando sin trabajo. Llega la última pregunta de la convivencia, y es más sutil de lo que parece. Cómo se jubila el sistema viejo sin perder lo que solo él sabía.

Capítulo 19

Jubilar el oráculo sin perder la referencia

El sistema viejo, y su réplica fiel que ha hecho de oráculo, han sido tu fuente de verdad durante toda la modernización. Cada vez que dudabas de qué debía hacer el sistema nuevo, se lo preguntabas al oráculo. Ahora el nuevo está al mando y el viejo sobra. La tentación es apagarlo y celebrarlo. Pero apagar el oráculo es perder para siempre la única fuente que sabía, con certeza, qué hacía el sistema viejo en cada caso, incluidos los que nunca capturaste. Hay que jubilarlo con cuidado, porque es un cuidado que no se puede repetir, una vez apagado, no hay vuelta atrás.

La regla de secuencia es tajante. **Captura todo lo que vas a necesitar del oráculo antes de jubilarlo, porque después no podrás re-derivarlo.** Mientras el oráculo vive, puedes hacerle cualquier pregunta nueva que se te ocurra y obtener la verdad. Cuando muere, la única referencia que queda es lo que capturaste, la especificación congelada. Si un caso no está en la especificación y el oráculo ya no existe, ese comportamiento se ha perdido como fuente de verdad, solo te queda lo que el sistema nuevo haga, que ya no puedes contrastar contra nada. Por eso la jubilación del oráculo se gatea por una condición, la especificación captura todo lo que vamos a necesitar, a la cobertura que hemos acordado que es suficiente. Mientras esa condición no se cumpla, el oráculo no se apaga.

Una vez capturado todo y jubilado el oráculo, la especificación congelada se convierte en la referencia permanente, y aquí se ve por qué insistimos tanto en su forma. La especificación, que era patrón de paridad mientras el oráculo vivía, pasa a ser la red de regresión del sistema nuevo, ahora solo con su pieza de traducción, sin la del viejo que ya no existe. Cada cambio futuro del sistema nuevo se comprueba contra esa especificación congelada, que garantiza que el comportamiento heredado se preserva. Y si alguna vez hay que cambiar deliberadamente una regla heredada, ese cambio se hace editando filas concretas de la especificación, de forma explícita y ratificada, no por accidente. La especificación es la memoria del sistema viejo, escrita de forma que sobrevive a su autor.

Hay un matiz práctico de coste que conviene aprovechar, y distingue dos jubilaciones que la gente confunde. Una cosa es jubilar el sistema viejo de verdad, el original, el que corría en la plataforma cara, y eso es caro y se hace en el cutover definitivo, liberando el coste que justificaba toda la modernización. Otra cosa, muy distinta, es jubilar la réplica fiel, el oráculo local, que corre barato en tu propia infraestructura. La réplica es barata de mantener encendida. Así que una estrategia prudente es retirar el sistema viejo original en cuanto el cutover lo permite, para capturar el ahorro, pero conservar la réplica fiel un tiempo más, como oráculo de reserva. Si tras el cutover aparece una sorpresa en producción, un caso que el sistema nuevo maneja de forma dudosa, puedes volver a preguntarle a la réplica qué habría hecho el viejo, y desambiguar. La réplica es el seguro barato que conservas después de apagar lo caro, y la jubilas tú también cuando la confianza acumulada hace que ya no merezca la pena.

Toca cerrar con el residuo honesto, porque este método no promete certeza absoluta y fingir que la promete sería traicionar todo lo dicho. Cuando el oráculo, original y réplica, ya no existe, la especificación es la única referencia, y la especificación es finita. Todo comportamiento del sistema viejo que no se capturó, las rutas que nunca se ejercitaron, los casos raros que ni el tráfico de sombra disparó, se ha vuelto irrecuperable de su fuente original. Si alguno de esos casos despierta algún día en producción, ya no hay oráculo al que preguntar qué era lo correcto, solo queda decidirlo de nuevo, ahora sí como diseño, no como recuperación. Entra por la puerta de los bugs y se trata como uno más: se reproduce, se decide qué debe hacer, se especifica y se cubre, como cualquier defecto que aparece en producción. Esto no es un fallo del método, es un límite de la realidad, ningún proceso puede capturar lo que nunca se manifestó. Lo que el método hace es acotar

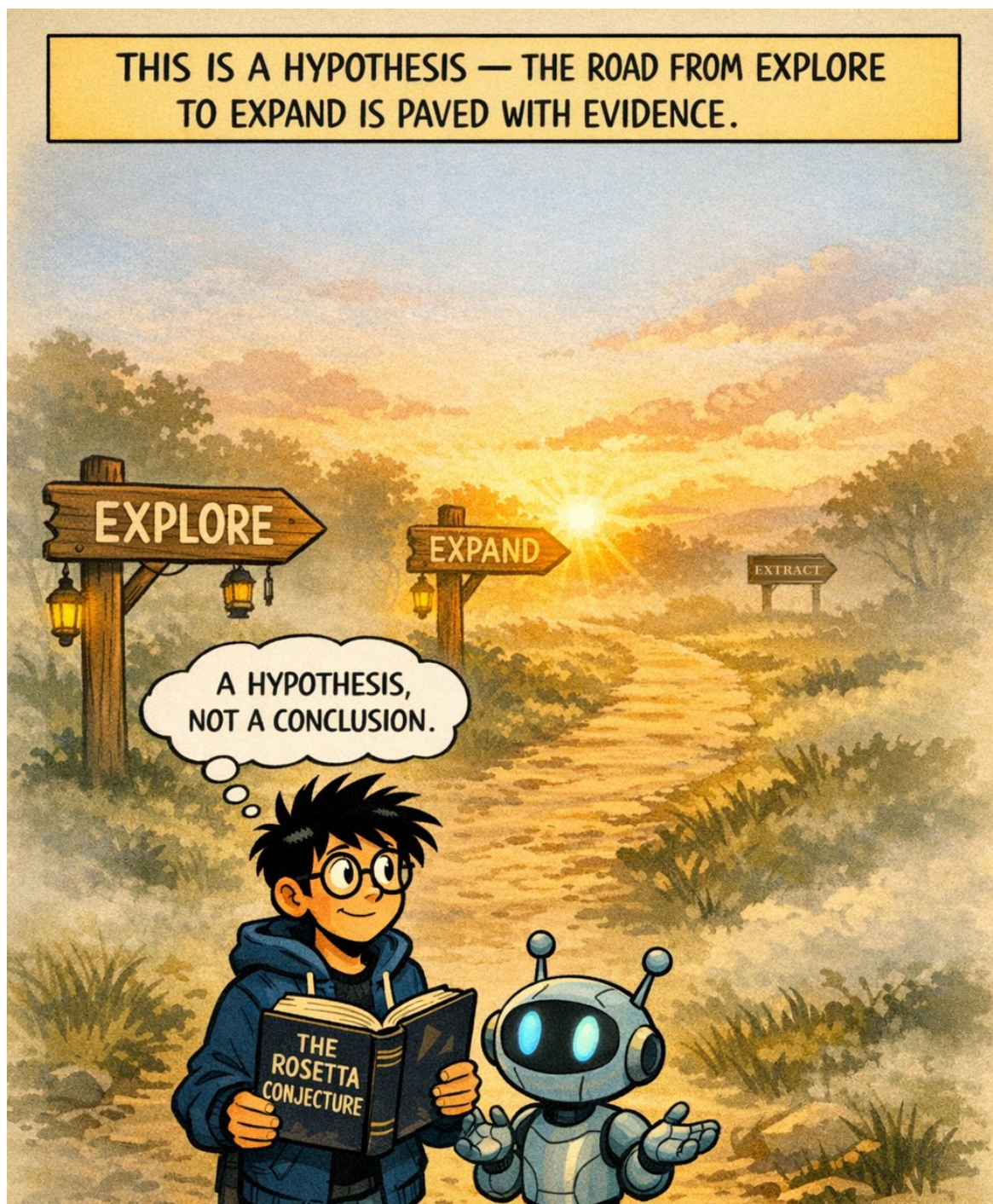
ese residuo y hacerlo visible, capturar amplio mientras el oráculo vive, correr la sombra sobre tráfico real para arrastrar a la luz los casos raros, y marcar honestamente los huecos que quedan. Después de jubilar el oráculo, la realidad es el único maestro que queda. El método se asegura de que llegues a ese punto habiendo aprendido del oráculo todo lo que se dejaba aprender.

Parte VI

De Explore a Expand

Capítulo 20

Esto es una hipótesis



Llegados aquí, conviene quitarse una máscara que muchos libros técnicos no se quitan nunca. Lo que acabas de leer no es un método probado contra cientos de migraciones de producción. Es una hipótesis, coherente y razonada, sobre cómo debería hacerse la modernización cuando se toma en serio que es un problema de dominio y se aprovecha que ahora hay agentes en todas las plantas del ascensor. Está, en los términos de Beck, en la fase Explore. Y decirlo no debilita el libro, lo fortalece, porque un argumento que se presenta como lo que es resiste mejor que una promesa que se presenta como lo que no es.

Vale la pena recoger por qué el método debería funcionar, porque esa es la sustancia de la hipótesis. Debería funcionar porque ataca la causa real del fracaso, confundir modernización con traducción, en vez de un síntoma. Debería funcionar porque se apoya en una fuente de verdad sólida, el comportamiento observable del sistema vivo, en vez de en documentación que miente o código que engaña. Debería funcionar porque preserva lo que de verdad importa, la semántica, y libera lo que de verdad estorba, la estructura vieja, en vez de confundir las dos cosas. Debería funcionar porque demuestra la equivalencia en vez de asumirla, y la demuestra por una cadena de pasos cada uno fácil de verificar. Y debería funcionar porque trata la convivencia y el cambio con el respeto que merecen, en vez de fingir que existe un interruptor. Nada de esto lo invento yo. Cada pieza se apoya en hombros conocidos que han sobrevivido a su propia prueba del tiempo: el Diseño Dirigido por el Dominio de Eric Evans, el ascensor del arquitecto de Gregor Hohpe, el 3X de Kent Beck, el estrangulador de Martin Fowler y el trabajo de Michael Feathers sobre cómo entender y cambiar sistemas heredados. Modernizar bien un sistema que importa siempre fue difícil. Lo que ha cambiado es que, con un agente en cada planta del ascensor, por fin es abordable. La apuesta del libro es que combinarlos así, con el dominio en el centro y el agente como augmentación en cada planta, produce algo más que la suma, un proceso que se gana la palabra modernización.

Pero una hipótesis no es una conclusión, y conviene ser claro sobre qué la convertiría en algo más. Pasar de Explore a Expand exige evidencia que ahora mismo no existe. Exige recorrer el método entero sobre encargos reales de producción, varios, distintos entre sí, y ver no solo que funciona sino entender por qué funciona lo bastante bien como para predecir cuándo no va a funcionar. Esa capacidad de predecir el fallo es la señal de madurez de Beck, la frontera entre una corazonada afortunada y un método. Hasta tenerla, la actitud honesta es seguir explorando, recorrer el ascensor en ciclos rápidos, aprender de cada uno, descartar sin pena lo que se revele equivocado, y resistir la tentación de congelar y productizar antes de tiempo.

Hay preguntas abiertas que el propio método no resuelve y que la fase Expand tendrá que abordar. ¿Cómo escala la recuperación de dominio cuando el sistema no es uno sino una constelación de sistemas enredados entre sí? ¿Qué pasa cuando el dominio recuperado revela que el negocio mismo ya no quiere hacer lo que el sistema viejo hacía, cuando la modernización destapa una oportunidad de cambiar las reglas y no solo de preservarlas? ¿Dónde está el límite real de lo que un agente puede recuperar con fiabilidad antes de que el juicio humano tenga que cargar con casi todo? ¿Cómo se gobierna todo esto cuando no hay un arquitecto sino muchos, con criterios distintos, sobre el mismo sistema? Ninguna de estas preguntas tiene aquí una respuesta cerrada, y fingir que la tiene sería volver a ponerse la máscara. Son el trabajo que queda.

Esa honestidad es, al final, coherente con el método mismo. Todo el libro predica que más vale demostrar que asumir, que la procedencia importa, que hay que marcar lo que no se sabe en vez de fingir certeza. Sería incoherente que el libro se eximiera a sí mismo de su propia disciplina y se presentara como verdad probada cuando es hipótesis razonada. Así que aquí está su procedencia, marcada como manda el método. Esto es un argumento desde principios, no un hallazgo de campo validado. Su fuerza está en la solidez del razonamiento y en la calidad de los principios que combina, no en una evidencia que todavía no existe.

Si el método es bueno, lo demostrará el recorrido, encargo a encargo, ciclo a ciclo, en la fase Explore donde ahora vive, hasta que la evidencia justifique escalarlo. Y si en ese recorrido se revela equivocado en alguna parte, la actitud correcta no será defenderlo, será aprender del fallo y rehacerlo, porque eso es exactamente lo que significa explorar. Este libro no es el final de una investigación. Es la mejor forma que he sabido dar, hoy, a una pregunta que sigue abierta. Cómo se moderniza de verdad un sistema que importa, poniendo el dominio en el centro, con un agente en cada planta del ascensor, y sin romper nada por el camino. No conozco otro intento de plantarse en esa intersección, el dominio, los agentes y la modernización, prometiendo solo lo que se puede demostrar. Si aparece uno mejor, bienvenido sea, porque la pregunta importa más que quién la responda. La invitación es a recorrerlo, no a creerlo.

El dominio dirige. El agente hace asequible la prudencia.

Apéndice A

Antipatrones

Los antipatrones son los errores que el método combate, recogidos aquí para reconocerlos rápido. Cada uno es la sombra de un principio del libro.

La traducción que se cree modernización. Transpilar el sistema viejo al lenguaje nuevo línea por línea y declarar el trabajo hecho. Conserva el diseño viejo y el mecanismo entretejido, ahora en sintaxis moderna y por tanto más difícil de detectar. Es el antipatrón raíz, el que da sentido a todo el libro.

Velocidad sobre un pipeline sin certificar. Acelerar con agentes una cadena de traducción que nunca demostró equivalencia de comportamiento. La velocidad aplicada a un método que no funciona no produce modernización, produce el siguiente post-mortem antes.

La recompilación que se cree destino. Hacer correr el sistema viejo sobre una plataforma nueva sin tocar su diseño, y creer que con eso se ha modernizado. Es una reubicación legítima disfrazada de transformación. El error no es recompilar, es creer que se ha terminado.

Matar el COBOL demasiado pronto. Reescribir o transpilar código que estaba estable, correcto y haciendo su trabajo, en vez de encapsularlo y dejarlo en paz. Gasta presupuesto y expertos escasos en lo que no lo pedía, y cambia opcionalidad por riesgo.

El refactor primero, la red después. Empezar a mejorar la estructura del sistema nuevo antes de haber demostrado su equivalencia con el viejo. Refactorizar sin red, sin nada que detecte si has roto el comportamiento. Viola la ley de orden, y es la raíz de los desastres de modernización más sonados.

El doble salto. Cambiar la plataforma y el diseño en el mismo movimiento. Cuando algo falla, y algo falla, no puedes diagnosticar si fue el cambio de plataforma o el de diseño. Pierdes la capacidad de aislar la causa justo cuando más la necesitas.

La elección disfrazada de hallazgo. Colar una decisión de diseño en el modelo recuperado como si fuera algo observado en el sistema viejo. Contamina el retrato del dominio, porque mezcla lo que el sistema hace con lo que tú quieres que haga. La disciplina de procedencia existe para detectarlo.

La ratificación que resuelve dudas. Convertir, en una reunión de ratificación, una conjetura dudosa en hecho aceptado, por consenso y sin nueva evidencia. La ratificación debe filtrar lo que tiene fuente firme, no bendecir lo dudoso con una firma.

La pérdida silenciosa de semántica. Descartar como mecanismo técnico algo que escondía una regla de negocio. El caso típico es el control de un error técnico que en realidad codificaba una decisión de dominio. Donde el mecanismo y la lógica se entretejen, ahí acecha este antipatrón, y solo el oráculo lo desambigua.

La paridad que compara de más. Incluir en la comparación de equivalencia detalles técnicos que el sistema nuevo tiene derecho a hacer distinto. Produce un mar de falsos positivos, la prueba se vuelve ruido, y el equipo acaba ignorándola, que es lo peor que le puede pasar a una prueba.

La paridad que compara de menos. Excluir de la comparación cosas que sí eran observables de dominio. Produce falsa seguridad, deja pasar cambios reales de comportamiento, y te invita a romper el sistema con la conciencia tranquila. Peor que el anterior, porque el silencio engaña más que el ruido.

El troceado que parte el estado. Migrar a trocitos un conjunto de operaciones que escriben el mismo dato, dejando unas en el sistema viejo y otras en el nuevo. El dato se parte en dos almacenes que divergen. Corrompe el estado más crítico del negocio sin avisar. Se evita cortando por la propiedad de la escritura.

El cutover con fe. Cortar al sistema nuevo porque las pruebas pasan, sin haber corrido en sombra sobre tráfico real ni haber resuelto las divergencias. Es afirmar la equivalencia en vez de demostrarla contra la realidad. La historia de los desastres de modernización está escrita en gran parte con este antipatrón.

Apagar el oráculo demasiado pronto. Jubilar el sistema viejo y su réplica antes de haber capturado todo lo que se iba a necesitar. Una vez apagado, no hay vuelta atrás, y el comportamiento no capturado se pierde como fuente de verdad para siempre.

Productizar en Explore. Tomar un método o un enfoque que funcionó una vez y congelarlo como estándar corporativo antes de entenderlo lo bastante para predecir cuándo fallará. Salta de Explore a Extract sin pasar por Expand. Un éxito no es un patrón.

Apéndice B

Glosario

Agente. Sistema capaz de leer gran cantidad de material, razonar sobre él y producir resultados. En este libro, una augmentación del arquitecto presente en todas las plantas del ascensor, que halla pero no elige.

Armazón (harness). La máquina de estados con guardas que organiza el trabajo del agente. Cada estado tiene su meta, su contexto limpio y su acceso a herramientas acotado, y una transición solo ocurre si se cumple una condición determinista.

Ascensor del arquitecto. Metáfora de Gregor Hohpe. El arquitecto sube y baja entre el penthouse, donde se decide la estrategia, y la sala de máquinas, donde el código se ejecuta, conectando plantas que de otro modo no se hablarían.

Cadena transitiva. Estrategia de verificación que evita comparar el sistema nuevo con el viejo en directo. Demuestra equivalencias intermedias fáciles, viejo con réplica fiel, réplica con especificación, especificación con sistema nuevo, y deja que la transitividad concluya la equivalencia final.

Capa anticorrupción (ACL). Patrón estratégico de DDD. Capa que un contexto consumidor levanta para traducir a su propio modelo el de cualquier otro, de modo que la forma ajena no lo contamine. En este libro su uso más frecuente es proteger el modelo nuevo de la forma del sistema viejo.

Contexto delimitado (bounded context). Frontera dentro de la cual un modelo del dominio y su lenguaje son coherentes. Las costuras entre contextos son donde más adelante se puede cortar.

Cutover. El corte. El momento en que una parte del sistema deja de atenderse en el viejo y pasa a atenderse en el nuevo de forma definitiva.

Diseño Dirigido por el Dominio (DDD). Enfoque que modela el software desde el dominio del negocio y su lenguaje, con conceptos como invariantes, agregados, contextos delimitados y lenguaje ubicuo.

Doble salto. Antipatrón de cambiar plataforma y diseño en el mismo movimiento. Prohibido porque destruye la capacidad de diagnosticar fallos.

Especificación ejecutable. Colección de casos en forma de dado esto, cuando aquello, entonces esto otro, escritos en lenguaje de dominio, que una máquina puede ejecutar y comprobar. Es el pivote de la cadena transitiva y vive tres vidas, paridad, regresión y documentación.

Fidelidad. Compromiso de preservar los invariantes y la semántica del sistema viejo, no su estructura. Lo que el sistema significa es sagrado, cómo lo consigue es libre.

GraphRAG. Emparejamiento de una búsqueda semántica que propone candidatos y un grafo determinista que los verifica, sobre una proyección del sistema. La búsqueda propone, el grafo dispone.

Invariante. Regla que el dominio debe cumplir siempre. Parte del espacio del problema, sobrevive a cualquier rediseño.

Lenguaje publicado (PL). Patrón estratégico de DDD. Lenguaje compartido y bien documentado, normalmente emparejado con un host abierto, que sirve de medio común en que dos contextos se comunican. Es el idioma del contrato, no el contrato mismo.

Lenguaje ubicuo. Vocabulario compartido de un contexto del dominio, en el que se expresa la especificación, independiente del idioma de cualquier sistema concreto. Es uno por contexto delimitado, no uno global. Su independencia es lo que protege la libertad de rediseño.

Mapa de contextos. Patrón estratégico de DDD. El plano de los contextos delimitados de un parque y de las relaciones entre ellos. En este método hay dos: el mapa as-is recuperado, que es dato, y el mapa objetivo, que es decisión.

Núcleo compartido (Shared Kernel, SK). Patrón estratégico de DDD. Subconjunto del modelo que dos contextos comparten y mantienen en común, con coordinación estrecha. Acopla, así que se reserva para lo que de verdad debe ser idéntico en los dos.

Cliente y proveedor (Customer/Supplier). Patrón estratégico de DDD. Relación entre dos contextos en la que el de aguas abajo es cliente y el de aguas arriba proveedor, y negocian prioridades y compromisos.

Conformista (Conformist). Patrón estratégico de DDD. El contexto de aguas abajo adopta el modelo del de aguas arriba tal cual, sin traducir. Barato y honesto cuando el modelo ajeno es bueno; lo contrario de levantar una capa anticorrupción.

Caminos separados (Separate Ways). Patrón estratégico de DDD. Decisión de no integrar dos contextos y duplicar lo poco que haga falta, cuando integrar cuesta más de lo que vale.

Mecanismo. La fontanería que hace funcionar la lógica sobre una plataforma concreta. Espacio de la solución vieja. Es lo que tienes derecho a cambiar, frente a la lógica de negocio, que debes preservar.

Modelo recuperado. Retrato fiel del dominio del sistema viejo, en términos de DDD, despojado de mecanismo, con cada afirmación marcada por su procedencia. Describe el estado actual del dominio, no el deseado.

Oráculo. El sistema viejo, o su réplica fiel, en tanto fuente de verdad sobre el comportamiento. Le das una entrada, te da la salida correcta por definición. Sustituye a la documentación y al recuerdo como fuente.

Paridad caja negra. Verificación de equivalencia que compara solo lo observable desde fuera, entradas y salidas de dominio, nunca las tripas, porque las tripas divergen a propósito.

Paridad de comportamiento. Lo que la verificación persigue: que el sistema nuevo signifique lo mismo que el viejo en lo observable de dominio. No es lo mismo que la paridad de datos, mover los datos intactos, que puede lograrse mientras el comportamiento se rompe.

Pipeline sin certificar. Cadena de transformación, hoy a menudo agéntica, que entrega código nuevo sin haber demostrado equivalencia de comportamiento contra la realidad. Es la crítica del libro a la industria, que se aplica a Rosetta misma hasta que la equivalencia se demuestra de extremo a extremo.

Procedencia. Marca que declara de dónde viene cada afirmación del modelo recuperado, y por tanto cuánto se puede fiar uno de ella. Extraído, inferido, atestiguado, sin probar. Lo que no puede citar fuente es una elección, no un hallazgo, y sale del modelo recuperado.

Réplica fiel. Copia del sistema viejo que preserva su estructura y corre rápido en local, hecha por ejemplo recompilando a una plataforma moderna. Sirve de oráculo barato y rápido. No confundir con el sistema nuevo, que diverge a propósito.

Señales (signals). Lo que el sistema emite sobre sí mismo mientras trabaja, con su procedencia, y que permite a las guardas decidir con fundamento. Incluye la cobertura, que pone número al residuo.

Servicio de host abierto (Open Host Service, OHS). Patrón estratégico de DDD. Protocolo o servicio público por el que un contexto ofrece sus capacidades a muchos consumidores a la vez, sin exponer su interior. Es la forma de encapsular el COBOL estable: se invoca, no se hereda.

Sobre el bucle (on-the-loop). Posición del arquitecto frente al almacén. No aprueba cada transición, eso lo hacen las guardas, sino que vigila la máquina y decide donde, por diseño, la máquina no decide. Frente a estar dentro del bucle.

Sombra. Modo de convivencia en que el tráfico real va a los dos sistemas a la vez, pero solo cuenta la respuesta del viejo. El nuevo procesa en paralelo sin afectar al negocio, y su respuesta se compara. Es la última prueba contra la realidad antes del cutover.

Substrato determinista. La proyección del sistema derivada mecánicamente, reproducible y con procedencia, sobre la que el agente razona y contra la que se verifica. No la inventa el agente.

Triage. Decisión de cartera, posterior a la recuperación, sobre qué hacer con cada parte del sistema según su tipo de subdominio: lo genérico a producto, lo de soporte y estable a encapsular, el núcleo a reescribir.

Estrangulador. Patrón de Martin Fowler. Una fachada delante de todo enruta cada petición al viejo o al nuevo, y va trasladando responsabilidades al nuevo poco a poco, hasta que el viejo se puede retirar.

3X. Modelo de Kent Beck. Explore, Expand, Extract. Tres fases en la vida de una idea, cada una con su mentalidad. Confundirlas es un error caro. Este método vive en Explore.